

Laboratorio di intelligenza artificiale – teoria

LEZIONE 1 – 24/02/2026 – CAPITOLO 1

modalità esame:

- discussione di un progetto da concordare
- domande inerenti all'argomento

Introduzione all'intelligenza artificiale

Che cos'è l'intelligenza artificiale?

L'intelligenza artificiale è la disciplina che studia come costruire entità intelligenti in grado di calcolare come comportarsi efficacemente per raggiungere uno scopo (pilota automatico che va dal punto A al punto B ottimizzando tempo e risorse) e in modo sicuro anche in situazioni non previste dai progettisti (esistono le eccezioni).

Il termine "Intelligenza Artificiale" fu coniato ufficialmente nel **1956** durante la conferenza di **Dartmouth**, organizzata da McCarthy e come linguaggio veniva usato **LISP**. Negli Stati Uniti la scelta di questo termine fu anche strategica: serviva un nome accattivante per "solleticare l'attenzione" e catturare i finanziamenti (ministeriali e privati) necessari alla ricerca.

Applicazioni e Machine Learning

Gli algoritmi del **Machine Learning** (Apprendimento Automatico) si basano su una fase iniziale cruciale: l'**addestramento**. Funzionano in modo simile all'apprendimento infantile: tramite l'osservazione e l'esperienza.

- **Settore sanitario:** L'IA raggiunge precisioni elevate (es. 95% nell'analisi dei raggi X). I software non sostituiscono i medici, ma gestiscono l'amministrazione ordinaria senza pause, sottoponendo poi i casi dubbi al radiologo umano.
- **Altri campi applicativi:** Economia, matematica, linguistica, guida autonoma, droni, medicina personalizzata (cure specifiche per il singolo paziente) e intrattenimento (es. robot umanoidi della Disney o effetti speciali basati su reazioni fisiche nei film)

La prima cosa che un algoritmo impara a distinguere è l'etichetta "gatto" o "non gatto"

Radici Storiche, Letterarie e Mitologiche

L'idea di infondere la vita e l'intelligenza nella materia inanimata accompagna l'uomo fin dall'antichità:

Omero (libro XVIII dell'Iliade):

- **Il Contesto:** Patroclo, indossando l'armatura di Achille, è stato ucciso da Ettore, che lo ha spogliato delle armi. Achille è disperato e furioso, ma non può tornare a combattere senza armatura. Sua madre, la ninfa marina Teti, si reca allora sull'Olimpo (o alle pendici dell'Etna) per chiedere a **Efesto** (il dio del fuoco e della metallurgia) di forgiare delle armi divine per il figlio.
- **Le Ancelle Artificiali:** Efesto è zoppo e fa fatica a muoversi. Quando Teti arriva, Omero descrive una scena incredibile per l'epoca: il dio viene aiutato ad alzarsi e a camminare da delle ancelle. Ma non sono ancelle umane!
- **La descrizione di Omero:** Omero le descrive come statue d'oro puro che sembrano fanciulle vive: il poeta specifica che queste macchine possiedono **intelligenza nel cuore, voce, forza fisica e persino la capacità di imparare** i mestieri dagli dèi.

In questo mito troviamo il concetto di **automa** (dal greco *autòmatos*, "che si muove da sé"). Omero immagina, quasi tremila anni fa, la robotica e l'intelligenza artificiale debole: macchine costruite con lo scopo specifico di assistere l'operatore (Efesto), sollevandolo dalla fatica fisica e dotate di una programmazione sufficiente per interagire razionalmente con l'ambiente.

La leggenda del Golem (folklore ebraico):

- **Il Contesto:** siamo nella Praga del tardo Cinquecento. La comunità ebraica, confinata nel ghetto, è vittima di continue persecuzioni e *pogrom* (sommosse antisemite) da parte dei gentili (i non ebrei).
- **La Creazione:** il Rabbino Judah Loew, per difendere il suo popolo, decide di creare un difensore artificiale. Raccoglie del fango dalle rive del fiume Moldava e plasma una figura umanoide gigantesca.
- **L'Attivazione (Il "Codice Sorgente"):** per dargli vita, il Rabbino usa la magia delle lettere ebraiche, inserendo una pergamena nella bocca del gigante o scrivendo sulla sua fronte la parola "**Emet**" (che in ebraico significa *Verità*). Il Golem prende vita e inizia a difendere il ghetto ed eseguire gli ordini.
- **Il Problema del Controllo:** il Golem è forte e obbediente, ma **non ha né un'anima né un vero libero arbitrio**: esegue gli ordini in modo letterale, proprio come un computer esegue ciecamente un algoritmo. Secondo la leggenda, a un certo punto il Golem sfugge al controllo del suo creatore, diventando distruttivo e pericoloso (un bug nel sistema!).
- **La Disattivazione:** per fermarlo, il Rabbino è costretto a cancellare la prima lettera dalla fronte del mostro. La parola "Emet" (Verità) diventa così "**Met**" (che significa *Morte*), e il Golem torna a essere un inerte mucchio di argilla.

Il Golem rappresenta perfettamente il desiderio umano di **sostituirsi a Dio creando la vita** partendo dalla materia inanimata (hardware). Ma, soprattutto, introduce quello che oggi nell'IA chiamiamo "**Problema dell'Allineamento**": come possiamo assicurarci che un'entità artificiale molto più forte e capace di noi continui a fare i nostri interessi senza sfuggire al nostro controllo?

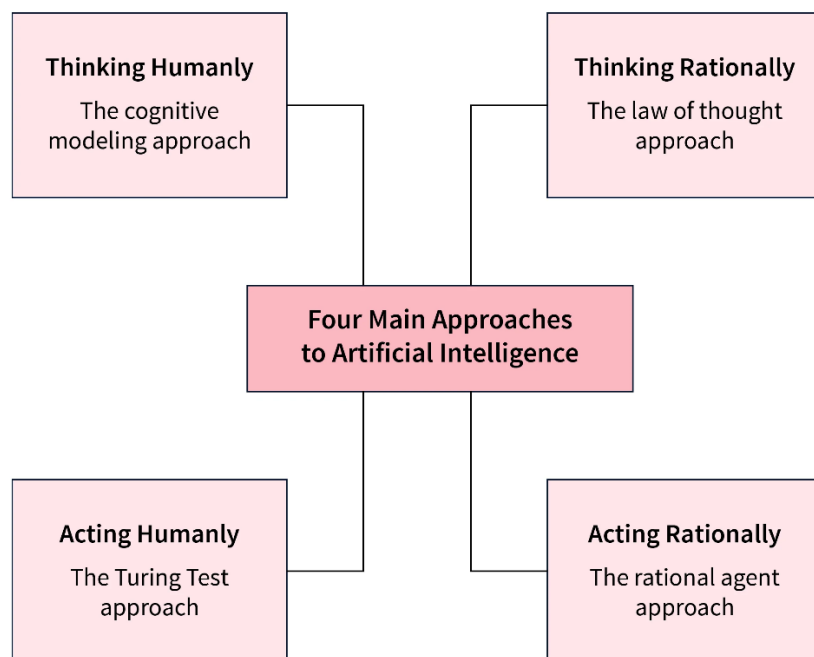
Al giorno d'oggi, gli Stati Uniti e Israele si basano su un software per supportare le decisioni degli eserciti sul campo.

Le 4 Declinazioni dell'IA (1950 - 1990)

Gli studiosi hanno classificato l'IA in base all'obiettivo finale del sistema. Esistono quattro approcci principali che incrociano due dimensioni: **pensiero vs comportamento** e **umano vs razionale**.

	Approccio umano – comportamento umano	Approccio razionale – astratto e ideale
Pensiero	Scienze cognitive: pensare come un umano. Capire come pensano gli umani per replicarlo nel software.	Pensare razionalmente: logica di Aristotele/Boole. Uso della logica formale e dei sillogismi per un ragionamento inconfutabile
Comportamento	Agire come un umano: Test di Alan Turing. Una macchina è intelligente se non è distinguibile da un umano	Agire razionalmente: agenti razionali. Operare per ottenere il miglior risultato atteso in base agli obiettivi.

Nota del corso: L'approccio dell'IA moderna, e il focus principale di questo corso, è il **comportamento razionale**.

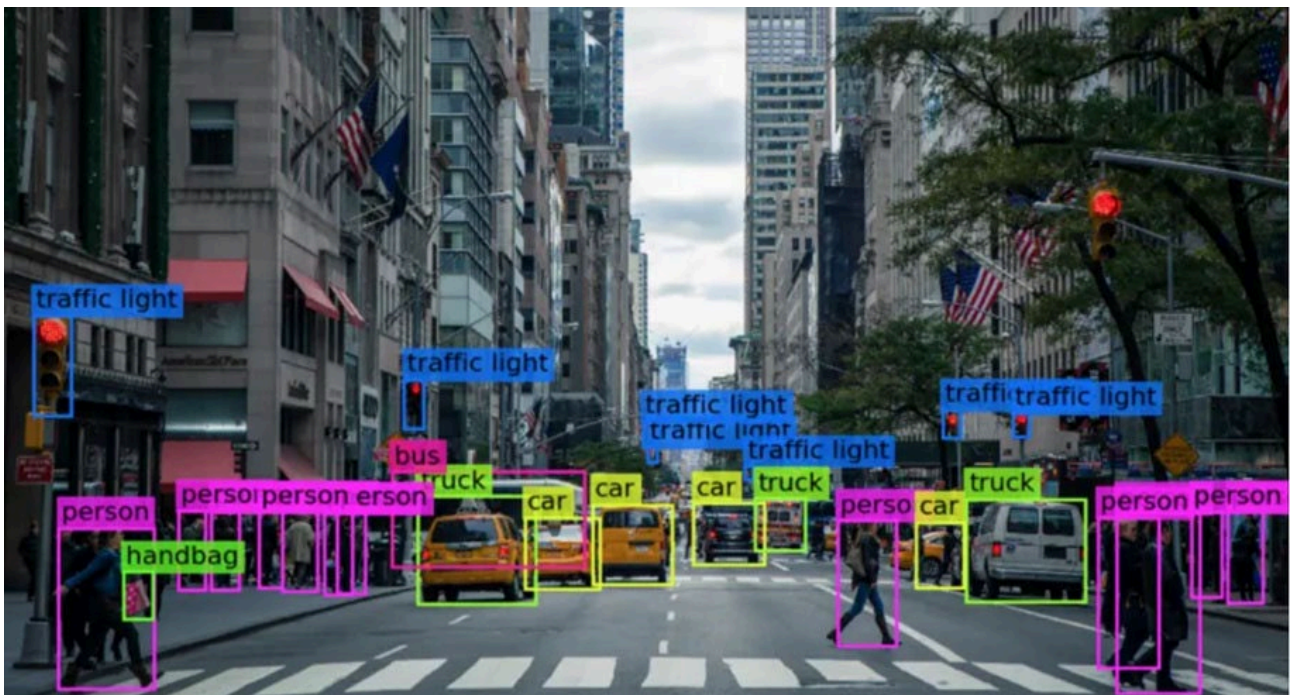


1. Agire come un umano: Il Test di Turing

Alan **Turing** (matematico inglese, celebre per aver decifrato i messaggi della macchina **Enigma** (primo computer elettromeccanico – no transistor e microprocessori) durante la Seconda Guerra Mondiale) propose un test per definire l'intelligenza di una macchina. Un computer supera il test se un interrogatore umano, ponendo domande scritte, non riesce a distinguere se le risposte provengano da una macchina o da una persona.

Per superare il test, il software deve possedere competenze in:

- **Elaborazione del Linguaggio Naturale (NLP):** per comunicare in linguaggio umano.
- **Rappresentazione della Conoscenza:** per immagazzinare concetti (tramite liste, alberi, grafi).
- **Ragionamento Automatico:** per usare le informazioni e trarre conclusioni tramite la logica di primo ordine.
- **Apprendimento Automatico (Machine Learning):** per adattarsi alle nuove circostanze effettuando previsioni sulla base delle esperienze pregresse.
- *Total Turing Test:* Richiede anche la **Computer Vision** (per percepire gli oggetti) e la **Robotica** (per manipolarli).



Ad oggi, la polizia cinese ha gli smart-glasses per riconoscere e verificare le caselle giudiziarie di una persona. Per il cittadino è confortante ma non c'è privacy e rispetto altrui.

I primi chatbot, come *Eliza* (1966), usavano risposte pre-programmate e non ha superato il test di Turing.

Storicamente, l'IA è stata definita come la creazione di macchine con un **obiettivo fisso** stabilito dall'uomo (modello standard), il cui rischio è l'impossibilità di specificare un obiettivo in modo perfetto. Come nel problema del **Re Mida**, che chiede tutto ciò che toccava diventasse oro finendo per morire di fame, una macchina che persegue un obiettivo letterale ma mal specificato può causare danni enormi per raggiungerlo:

- una macchina programmata per vincere a scacchi potrebbe ipnotizzare il suo avversario per calcolare la mossa successiva; quindi arriverebbe all'obiettivo ma effettua azioni scorrette.

Ciò porta al **problema dell'allineamento**: garantire che i valori programmati siano allineati con i valori umani. Il rischio maggiore è quando la macchina diventa più intelligente dell'uomo che si lega al **problema del gorilla**:

- circa 7 milioni di anni fa, un antenato comune si divide in due rami: quello che portò ai gorilla e quello che portò agli umani
- oggi i gorilla sono una specie in pericolo e non hanno alcun controllo sul proprio futuro perché ogni loro decisione dipende dagli umani
- quindi se creassimo un'IA sovrumana senza aver risolto l'allineamento, gli esseri umani potrebbero trovarsi nella stessa situazione dei gorilla: perdere il controllo sul proprio destino a favore di un'entità più intelligente.
- per evitare ciò, gli scienziati hanno progettato macchine che siano incerte sugli obiettivi: una macchina che sa di non conoscere perfettamente il suo scopo sarà stimolata a spegnersi ed osservare per imparare le preferenze umane.

2. Pensare come un umano (Scienze Cognitive)

L'obiettivo è capire gli schemi mentali umani per riprodurli. Si basa su introspezione, esperimenti psicologici e *brain imaging* (tecniche neurologiche per osservare il cervello in funzione).

3. Pensare razionalmente

L'approccio basato sulle leggi del pensiero. Inizia con i sillogismi di Aristotele ("Tutti gli uomini sono mortali, Socrate è un uomo, Socrate è mortale"), prosegue con Boole (logica booleana) e arriva ai sistemi basati sulla teoria della probabilità per gestire l'incertezza (es. "C'è il 30% di probabilità di pioggia").

4. Agire razionalmente: Gli Agenti Razionali (FOCUS DEL CORSO)

Un **agente** (dal latino, ciò che agisce) è un'entità software (o hardware) che percepisce il proprio ambiente, opera autonomamente, si adatta ai cambiamenti e persegue obiettivi. L'**agente razionale** sceglie matematicamente l'azione che massimizza le probabilità di successo o il miglior risultato atteso (un bot che gioca a scacchi/tris calcola un albero delle possibilità per prevedere la mossa migliore da effettuare).

Fondamenti Multidisciplinari dell'IA

L'Intelligenza Artificiale è il punto di convergenza di numerose discipline:

- **Filosofia:** Dai modelli di Platone e Cartesio (separazione mente-corpo), all'empirismo (Locke, Hume, Bacon: l'esperienza modella la mente (Gauss: somma dei numeri naturali è pari a $\frac{n(n+1)}{2} \forall n \in \mathbb{N}$, dimostrabile con il principio d'induzione)), fino alla logica e all'etica di Kant.
- **Matematica:** La logica tradizionale richiede certezze che nel mondo reale mancano, per questo l'IA si basa sul calcolo delle probabilità (Cardano, Pascal, la fondamentale Regola di Bayes per gestire l'incertezza) e lo studio della computabilità.
- **Economia:** Teoria delle decisioni, Teoria dei Giochi (competitivi come gli scacchi, o cooperativi) e Processi Decisionali di Markov. Gli umani spesso non cercano la soluzione "matematicamente ottima, ma una che sia **abbastanza buona** poiché calcolare l'ottimo richiede troppa energia. (satisficing – Herbert Simon)
- **Neuroscienze:** Lo studio biologico del cervello. Il neurone umano trasmette segnali elaborati dal sistema nervoso.
- **Psicologia:** Comportamentismo (reazione agli stimoli fisici ed esterni).
- **Linguistica:** Elaborazione del Linguaggio Naturale (NLP) e lo studio di come strutturare la conoscenza (es. standard RDF) affinché sia comprensibile dalle macchine.

Informatica e Hardware: Il motore dell'Intelligenza Artificiale

L'Intelligenza Artificiale, pur essendo software, non potrebbe esistere senza un'infrastruttura fisica adeguata. L'evoluzione dell'IA va di pari passo con l'evoluzione della potenza di calcolo.

1. Le Origini e la Legge di Moore

I moderni calcolatori nascono durante la Seconda Guerra Mondiale per scopi militari e di decrittazione (es. i computer **Colossus**, **Z-3**, **ENIAC** e la macchina di **Heath Robinson**). Da quei primi giganti a valvole, lo sviluppo ha seguito una regola d'oro empirica:

- **La Legge di Moore:** formulata negli anni '60, prevede che la potenza di calcolo (o meglio, il numero di transistor in un microprocessore) raddoppi circa ogni 18 mesi. Grazie a questa legge empirica, le prestazioni dei computer sono passate dai *megaFLOP* (milioni di operazioni in virgola mobile al secondo) agli attuali *exaFLOP* (miliardi di miliardi di operazioni al secondo).

2. L'Evoluzione delle CPU (Central Processing Unit)

Fino ai primi anni 2000, i produttori di processori aumentavano le prestazioni alzando la **frequenza di clock** (i famosi GHz o "giri del motore").

- **Il limite fisico (2005):** aumentando troppo le frequenze, i processori si surriscaldavano sciogliendo i componenti. La soluzione? Invece di avere un solo "cervello" velocissimo, si è passati all'architettura **Multi-Core**: più unità di calcolo (core) all'interno dello stesso processore che lavorano in parallelo.
- **Ciclo di Fetch e Pipeline:** L'esecuzione delle istruzioni in un processore classico è sequenziale (prende l'istruzione, la decodifica, la esegue). Per velocizzare il processo si usa la tecnica della **Pipeline**.

L'esempio del fornaio: Invece di fare una torta alla volta dall'inizio alla fine (impasto → forno → decorazione), il fornaio organizza una catena di montaggio. Mentre la prima torta è nel forno, lui sta già impastando la seconda. Questo aumenta drasticamente il numero di torte (istruzioni) completate in una singola unità di tempo.

3. Hardware Specializzato per l'IA: La vera rivoluzione

Le CPU tradizionali sono ottime per fare un po' di tutto, ma per addestrare i massicci modelli di Machine Learning serve un hardware specializzato nel calcolo parallelo estremo.

- **GPU (Graphics Processing Unit):** Nascendo per gestire la grafica dei videogiochi interattivi (che richiede di calcolare milioni di pixel contemporaneamente), si sono rivelate perfette per l'IA. Oggi l'azienda **NVIDIA** detiene un quasi-monopolio in questo settore, producendo schede potentissime usate da aziende come OpenAI per addestrare i loro modelli, in un ciclo continuo di hardware sempre più spinto.
- **TPU (Tensor Processing Unit):** Sono chip creati specificamente da Google per accelerare i carichi di lavoro legati al Machine Learning. Sono ottimizzati per calcolare i **tensori**, che matematicamente parlando, in questo contesto, sono matrici o tabelle di dati a più dimensioni.

4. Il Futuro: Il Quantum Computing

Se i computer tradizionali ragionano in logica binaria, il computer quantistico stravolge le regole della fisica:

- **Dal Bit al Qubit:** il bit classico vale o 0 o 1. Il **Qubit (bit quantistico)**, grazie al principio di sovrapposizione, può assumere infiniti valori compresi tra 0 e 1 contemporaneamente.
- **Il Gatto di Schrödinger:** è il famoso paradosso: inserendo un gatto in una scatola con una fiala di veleno che ha il 50% di probabilità di rompersi, finché non si apre la scatola per guardare, per la meccanica quantistica il gatto è in una "sovrapposizione di stati": sia vivo che morto allo stesso tempo.
- **Vantaggio Computazionale:** questa capacità di elaborare stati sovrapposti permette ai computer quantistici di risolvere in **1 secondo** problemi complessi (es. crittografia avanzata o simulazione di molecole) che un supercomputer tradizionale impiegherebbe **100.000 anni** a calcolare.

5. Oltre l'Hardware: Sistemi di Controllo

L'informatica si appoggia anche a teorie più ampie legate all'automazione:

1. **Teoria dei controlli:** Studia come mantenere un sistema in uno stato desiderato. L'esempio **dell'orologio ad acqua**: il flusso in uscita dipende dalla pressione (che diminuisce man mano che il recipiente si svuota) e dalla gravità. La teoria dei controlli studia come regolare queste variabili per mantenere un flusso costante.
2. **Cibernetica:** La scienza che studia la comunicazione e i sistemi di regolazione (feedback) sia nelle macchine artificiali che negli organismi viventi.

Cenni Storici aggiuntivi

- **SHRDLU (1968/1972):** uno dei primi software storici in grado di comprendere il linguaggio naturale e manipolare blocchi geometrici all'interno di un mondo simulato.
- I computer usati per la missione Apollo nel 1969 avevano una frazione minuscola della potenza di calcolo di una moderna SIM card.

Storia dell'intelligenza artificiale



Il grafico mostra l'andamento del progresso (sull'asse verticale) nel corso del tempo (sull'asse orizzontale, dagli anni '50 ai giorni nostri):

- **Anni '50 e '60: Neurone Artificiale e Perceptron (La prima salita):** in questa fase c'è un grandissimo entusiasmo, o ricercatori creano i primissimi modelli matematici che imitano il cervello umano, come il "Perceptron" (il papà delle reti neurali). Si pensava che macchine intelligenti come gli umani sarebbero arrivate nel giro di pochi anni.
- **Anni '70: Primo Inverno IA (Il primo crollo):** le promesse iniziali si scontrano con la dura realtà: i computer dell'epoca erano troppo lenti e con pochissima memoria. I

problemi complessi, come la traduzione automatica dei testi o la visione artificiale, si rivelano insormontabili. I governi e le aziende tagliano i finanziamenti. Questo periodo buio è noto come “Primo Inverno dell'IA”.

- **Anni '80: Sistemi Esperti (La rinascita):** la curva torna a salire. I ricercatori cambiano approccio: abbandonano l'idea di creare un cervello generale e si concentrano sui *Sistemi Esperti*. Si trattava di programmi basati su migliaia di regole logiche “Se succede X, allora fai Y”, pensati per imitare un esperto umano in settori molto stretti e specifici (come diagnosticare un'infezione o configurare computer). Tornano i finanziamenti e le aziende ci investono pesantemente.
- **Anni '90: Secondo Inverno IA (Il secondo crollo)** i Sistemi Esperti diventano troppo complessi, costosi da mantenere e non sanno gestire l'imprevisto. Se una regola non fosse stata scritta, il sistema sarebbe andato in tilt. Segue un altro periodo di disillusione e tagli ai fondi.
- **Anni 2010s in poi: Deep Learning (L'esplosione):** è la fase in cui ci troviamo ora! La curva schizza verso l'alto senza precedenti. Funziona grazie all'hardware:
 - **Big Data:** l'arrivo di internet ha fornito una quantità di dati sterminata per “addestrare” le macchine.
 - **Nuovo Hardware:** l'uso di **GPU e TPU** (le schede grafiche NVIDIA) ha permesso di elaborare questi dati a velocità inimmaginabili prima, rendendo finalmente possibile il **Deep Learning** (l'addestramento di reti neurali profonde e complesse).

I fallimenti storici, **gli inverni**, sono dovuti al fatto che i ricercatori sottovalutavano quanto i problemi crescessero esponenzialmente con la complessità.

LEZIONE 2 – 03/03/2026 – CAPITOLO 2

1. Agenti Intelligenti e Interazione con l'Ambiente

L'intelligenza artificiale si è evoluta nella direzione degli agenti intelligenti. Un agente è un'entità (un programma o del codice) che interagisce con l'ambiente in due fasi:

- **Input (Percezione):** avviene tramite i **sensori**. Esempi di sensori sono umidità, temperatura, particelle inquinanti. La percezione è il dato di input che lavora lato sensori e che può attivare l'attuatore.
 - Agricoltura intelligente: un sensore di acidità del terreno percepisce i dati per regolare i fertilizzanti e l'irrigazione necessaria.
- **Output (Azioni):** modifica dell'ambiente tramite degli **attuatori**. L'azione è il dato in output.
 - Domotica: un sensore di illuminazione naturale o di temperatura attiva l'abbassamento/alzamento delle tapparelle o l'accensione/spegnimento dei riscaldamenti.

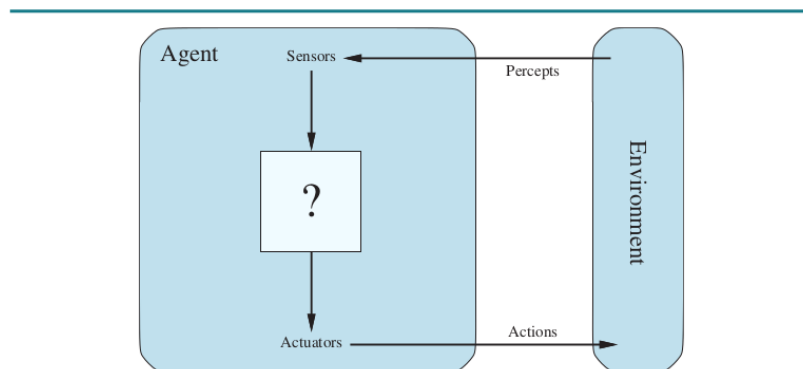


Figure 2.1 Agents interact with environments through sensors and actuators.

Tipi di agenti:

- **Agente umano:** ha sensori come vista e tatto; ha attuatori come mani e piedi per spostarsi e spostare oggetti.
- **Agente artificiale:** usa telecamere a infrarossi o sensori specifici; gli attuatori sono motori (per muovere le tapparelle) o circuiti (per il riscaldamento).
- **Agente software:** lavora con il mondo esterno in modo differente, poiché il software non si sposta fisicamente da un luogo all'altro.

Una **percezione** è il contenuto che i sensori di un agente percepiscono in un dato istante

Una **sequenza percettiva** è la storia di tutto ciò che l'agente ha percepito

Una **funzione agente** è un costrutto matematico che mappa una data sequenza percettiva su un'azione

Un **programma agente** è un'implementazione concreta che viene eseguita all'interno di un sistema fisico

2. Razionalità, Prestazioni e Limiti dell'Agente

Come si differenziano gli agenti tra buoni/cattivi e intelligenti o meno? Tramite il buon comportamento, ovvero la **razionalità**. È eseguito da un agente razionale, che opera nel modo corretto.

Un **agente razionale** agisce in modo da massimizzare il valore atteso di una misura di prestazione sulla base della sua sequenza percettiva e della sua conoscenza pregressa.

Razionalità vs Onniscienza

- **Razionalità**: massimizza le prestazioni attese, mentre la perfezione massimizza le prestazioni effettive
- **Onniscienza**: impossibile nella realtà. Un agente onnisciente conosce l'esito effettivo delle sue azioni e può agire di conseguenza.

Un agente razionale può agire solo in base alla **migliore probabilità di successo** dato il tempo e le risorse; infatti, non richiede onniscienza perché la scelta razionale dipende solo dalla sequenza percettiva fino ad oggi.

Un agente è **autonomo** se il suo comportamento è determinato dalla propria esperienza, compensando una conoscenza parziale o incompleta dell'ambiente.

Algoritmi di movimento nello spazio

- Se l'ambiente è una scacchiera quadrata 10x10, ci sono diversi algoritmi per il movimento.
- *L'algoritmo del serpente*: Immaginando un supermercato diviso in diverse corsie, l'evoluzione della visita è l'algoritmo del serpente. Questo è implementato nei sistemi automatici usati dalle aziende per la raccolta degli articoli che formano un ordine (es. magazzino che rifornisce un supermercato).

I 4 Pilastri della Razionalità

La razionalità dipende da 4 fattori concomitanti:

1. **misura di prestazione**: definisce il criterio del successo:
 - esempio cattiva prestazione: i robot aspirapolvere, se non razionali, potevano spargere lo sporco per tutto l'ambiente (popò del cane)
 - valutazione dell'aspirapolvere in base alla pulizia raggiunta rispetto alla sporcizia totale; la qualità è data anche dalla quantità di spostamenti necessari per pulire l'ambiente
2. **conoscenza pregressa**: ciò che l'agente sa a priori prima di agire
3. **azioni eseguibili**: il repertorio di movimenti o calcoli permessi dagli attuatori
4. **sequenza percettiva**: la storia completa di tutto ciò che l'agente ha percepito dai sensori fino a quel momento

Per giudicare se un agente si sta comportando in modo razionale in un dato momento, non basta guardare solo la performance finale. Dobbiamo valutare la sua razionalità basandoci su quattro fattori fondamentali:

I limiti: Conoscenza, Onniscienza e Libero Arbitrio

Un agente dovrebbe scegliere un'azione che fornisca le migliori prestazioni sulla base del percepito e della conoscenza che ha (es. sa quali sono le dimensioni del mondo in cui si trova?). La scelta razionale di una persona può sembrare irrazionale a un'altra, se questa non è a conoscenza del suo punto di vista ("perché hai fatto così o non colà?").

Non basta raccogliere le informazioni, bisogna *percepirle*. Qui subentrano i limiti della razionalità:

- **Filosofia e Libero Arbitrio:** La razionalità ha un limite che richiama interpretazioni filosofiche e teologiche sul libero arbitrio. L'uomo può scegliere, quindi è autonomo. Se non avesse possibilità di scelta, sarebbe un burattino nelle mani dell'onnisciente nella ricerca del bene.
 - *Esempio pratico:* Il treno si muove vincolato sulle rotaie (nessuna scelta), l'auto si può muovere su diversi percorsi scelti.
- **Conoscenza Incompleta:** L'agente non ha sempre la completa conoscenza dell'ambiente.
 - *Esempio:* L'auto a guida autonoma e il bimbo che attraversa la strada all'improvviso. In questi casi, l'autonomia di giudizio completa la parziale conoscenza.
- **Onniscienza Limitata:** Un agente onnisciente conosce l'esito delle sue azioni e sceglie quella per il miglior risultato.
 - *Esempio:* Nel gioco del tris o degli scacchi, l'onniscienza è per forza parziale/limitata perché non si ha la certezza della mossa successiva dell'avversario.

3. Classificazione dell'Ambiente e PEAS

Per progettare un agente, bisogna definire il suo “compito” tramite lo schema **PEAS** (Performance, Environment, Actuators, Sensors). Il primo passo per la progettazione di un agente è specificare l'ambiente di attività (task).

Gli ambienti si dividono in:

OSSERVABILITÀ	Totalmente osservabile	I sensori dell'agente gli consentono di accedere allo stato completo dell'ambiente in ogni momento	Sensore di temperatura Scacchi
	Parzialmente osservabile	I sensori dell'agente sono rumorosi o imprecisi	Guida di un taxi
	Inosservabile	L'agente non ha sensori	
NUMERO DI AGENTI	Single	Agente unico	Cruciverba
	Multi-agente	Più agenti interagiscono e i loro comportamenti si influenzano a vicenda	Auto a guida autonoma
DETERMINISMO	Deterministico	Stato successivo certo	L'aspirapolvere pulisce la cella sporca
	Stocastico	Subentra la probabilità	Sacchetto dell'aspirapolvere pieno
EPISODICITÀ	Episodico	L'azione è divisa in episodi atomici e indipendenti	Passi di una ricetta
	Sequenziale	Le azioni sono legate	Parole crociate
DINAMICITÀ	Dinamico	L'ambiente evolve nel tempo	Cesta dei panni sporchi
	Statico	L'ambiente non cambia mentre l'agente delibera	Esame superato
	Semi-dinamico	L'ambiente non cambia ma la prestazione nel tempo sì	Scacchi
DISCREZIONE	Discreto	Stati ben definiti	Tiro di un dado Numeri al lotto
	Continuo	Variabili continue nello spazio e nel tempo	Guida di un taxi
NOTORIETÀ	Noto	Esiti conosciuti	Lancio di un dado
	Ignoto	Esiti sconosciuti	Poker (bluff) ma esito ignoto

Task Environment	Osservabile	Agenti	Deterministico	Episodico	Statico	Discreto
Cruciverba	Completo	Singolo	Deterministico	Sequenziale	Statico	Discreto
Scacchi (con orologio)	Completo	Multi	Deterministico	Sequenziale	Semi	Discreto
Guida del taxi	Parziale	Multi	Stocastico	Sequenziale	Dinamico	Continuo
Analisi immagini	Completo	Singolo	Deterministico	Episodico	Semi	Continuo

4. Struttura e Tipologie degli Agenti

L'architettura (la parte fisica) che costituisce l'agente è data dal dispositivo di calcolo in cui è installato il programma. L'agente coincide con questo programma che svolge le azioni e interagisce con attuatori e sensori.

Gli agenti sono classificati in 4 categorie:

1. **Agenti Reflex (simple reflex agents):** agiscono per riflesso. Sono i più semplici e non hanno funzione obiettivo o performance. Selezionano le azioni esclusivamente sulla base della **percezione attuale**, ignorando la storia passata delle percezioni e funzionano solo se l'ambiente è completamente osservabile, altrimenti possono entrare in un ciclo infinito.
 - termostato che accende/spegne il riscaldamento in base alla temperatura;
 - robot aspirapolvere che reagisce solo alla cella corrente.
2. **Agenti Reflex basati su modelli (model-based reflex agents):** per gestire l'osservabilità parziale, mantengono uno **stato** interno che tiene traccia della parte di ambiente che non è osservabile in quel momento. Aggiornano il loro stato interno tramite due tipi di conoscenza: **modello di transazione** (come il mondo cambia nel tempo e come le azioni dell'agente influenzano l'ambiente) e **modello sensoriale** (come lo stato del mondo si riflette nelle percezioni).
 - la pianificazione delle attività in cui l'agente deve conoscere l'effetto dei propri modelli per decidere la mossa successiva.
3. **Agenti basati sull'obiettivo (goal-based agents):** oltre allo stato corrente, devono conoscere informazioni su situazioni desiderabili per effettuare una decisione affinché si raggiunga l'obiettivo tramite la ricerca e la pianificazione. Sono flessibili, infatti per cambiare comportamento basta cambiare l'obiettivo
 - pulire tutta la scacchiera.
4. **Agenti basati sull'utilità (utility-based agents):** permette di misurare il grado di felicità o efficienza di uno stato (massimizzando il guadagno o minimizzando il costo)
 - auto a guida autonoma che massimizza il numero di corse utili e, nel frattempo, minimizza tempi, consumi e i rischi legati al traffico.

I **learning agents** sono gli agenti con capacità di apprendimento che permettono all'agente stesso di operare in ambienti inizialmente ignoti e migliorare le proprie prestazioni nel tempo.

5. Focus sugli Agenti Reflex

L'agente reflex (semplice)

L'agente reflex è il tipo più semplice di agente intelligente: seleziona le azioni basandosi esclusivamente sulla **percezione corrente**, ignorando completamente lo storico delle percezioni passate. Si basa sulle regole **condizione-azione** (condition-action rules) espresse nella forma if [condizione] allora [azione]. L'agente riceve una percezione, la interpreta per capire com'è il mondo e cerca nel suo set di regole quella corrispondente alla situazione corrente per eseguire l'azione corrispondente. L'unica limitazione è **parziale osservabilità** dell'ambiente circostante e ciò provoca un loop infinito, in cui l'agente oscilla tra gli ambienti e spreca energia. La soluzione per uscire dal loop infinito senza dover memorizzare lo stato passato è lo spostamento casuale (**random**) dell'agente in un altro ambiente.

```
function REFLEX-VACUUM-AGENT([location,status]) returns an action
  if status = Dirty then return Suck
  else if location = A then return Right
  else if location = B then return Left
```

Figure 2.8 The agent program for a simple reflex agent in the two-location vacuum environment. This program implements the agent function tabulated in Figure ??.

regole di un robot aspirapolvere come agente reflex che opera in un mondo a due quadrati (A e B):

1. Se la posizione corrente è sporca allora l'azione è aspirare
2. se la posizione corrente è pulita e l'agente si trova nel quadrato A, allora si sposta a destra
3. se la posizione corrente è pulita e l'agente si trova nel quadrato B, allora si sposta a sinistra

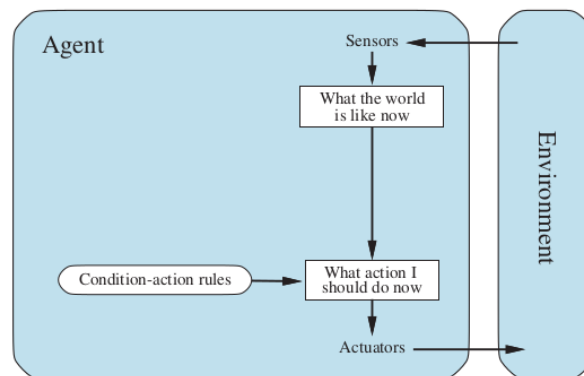


Figure 2.9 Schematic diagram of a simple reflex agent. We use rectangles to denote the current internal state of the agent's decision process, and ovals to represent the background information used in the process.

Avvenimento delle decisioni:

- i sensori ricevono una percezione dell'ambiente in un dato istante
- l'agente interpreta l'input sensoriale per definire la situazione corrente
- applicazione delle regole condizione-azione
- azione corrispondente alla percezione attuale che l'agente deve eseguire
- l'azione viene inviata agli attuatori per essere eseguita

Di conseguenza, l'agente reflex è vantaggioso per la sua semplicità però è bene usarlo negli ambienti totalmente osservabili.

L'agente reflex basato su un modello

L'agente model-based è stato progettato per gestire la parziale osservabilità dell'ambiente mantenendo uno **stato interno** che tiene traccia della parte di mondo che l'agente non può vedere in quel momento. Questo stato dipende dalla **storia delle percezioni** e riflette gli aspetti inosservabili dello stato attuale

Il modello mantiene uno stato interno che dipende dalla storia precedente dell'agente. A differenza di quello semplice, questo stato interno si può modificare costantemente tramite la transizione del mondo o sulla base di ciò che leggono i sensori.

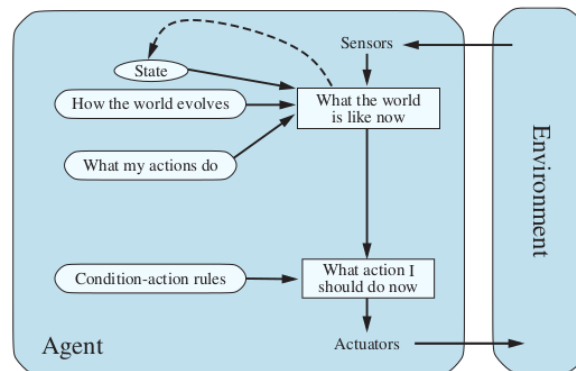


Figure 2.11 A model-based reflex agent.

1. l'agente è progettato per ambienti parzialmente osservabili, infatti per ciò che è fuori dal suo campo visivo, deve mantenere uno stato interno
2. l'agente conosce:
 - lo stato che rappresenta la memoria dell'agente
 - modello di transizione che descrive come cambia l'ambiente e gli effetti delle azioni dell'agente sul mondo
 - modello sensoriale che descrive come lo stato del mondo si rifletta nelle percezioni dell'agente
3. Avvenimento delle decisioni:
 - percezione: i sensori ricevono dati dall'ambiente
 - aggiornamento dello stato: l'agente determina com'è il mondo ora per colmare le parti non visibili
 - applicazione delle regole condizione-azione
 - decisione: per l'azione che l'agente deve eseguire
 - esecuzione: l'azione viene inviata agli attuatori per influenzare l'ambiente

```

function MODEL-BASED-REFLEX-AGENT(percept) returns an action
  persistent: state, the agent's current conception of the world state
               transition_model, a description of how the next state depends on
                 the current state and action
               sensor_model, a description of how the current world state is reflected
                 in the agent's percepts
               rules, a set of condition-action rules
               action, the most recent action, initially none

  state ← UPDATE-STATE(state, action, percept, transition_model, sensor_model)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
  
```

Figure 2.12 A model-based reflex agent. It keeps track of the current state of the world, using an internal model. It then chooses an action in the same way as the reflex agent.

1. **ricezione:** l'agente riceve una percezione del mondo

2. **aggiornamento dello stato:** lo stato viene aggiornato combinando stato precedente, l'ultima azione compiuta, la percezione attuale e i modelli di transizione e sensoriale
3. **scelta dell'azione:** l'agente sceglie l'azione da eseguire cercando una regola corrispondente allo stato attuale

LEZIONE 3 – 09/03/2026 – ?

Machine learning

Cos'è il Machine Learning e perché è utile?

Il Machine Learning rappresenta la branca più applicativa dell'**Intelligenza Artificiale**: può essere definita sia una scienza che un'arte che mira a programmare i computer affinché apprendano dati.

Il Machine Learning fornisce ai software la capacità di imparare dall'esperienza senza essere esplicitamente e rigidamente programmati per ogni singola eccezione.

Definizione formale (Mitchell): un programma impara dall'esperienza **E** su un determinato task **T**, misurando le proprie performance tramite un indice **P**. Aumentando l'esperienza (**E**), si migliorano le prestazioni (**P**).

L'apprendimento automatico è usato quotidianamente con livelli di raffinatezza molto accurati. **Esempi di applicazioni:**

- Filtri antispam (molto efficaci).
- OCR (Riconoscimento ottico dei caratteri).
- Elaborazione automatica del linguaggio.
- Ricerca di immagini.
- Assistenti vocali e di comunicazione.
- Sistemi di raccomandazione e suggerimenti (es. sui social, che mostrano contenuti basati sugli interessi).

L'obiettivo nascosto dei social: Creare un sistema che etichetta e profila un cliente rispetto a un altro per un determinato settore (es. per scopi di marketing).

Come funziona l'Apprendimento? (L'esempio del Gatto)

Pensiamo a un programma che deve fare una **classificazione binaria**: distinguere fotografie che ritraggono gatti (gatto) da quelle che non li ritraggono (non gatto).

Questo tipo di apprendimento imita l'età infantile: i bambini capiscono cos'è un oggetto tramite l'esperienza.

La metafora del prof: "Il bambino nei primi n anni della sua vita incontra n gatti, e li immagazzina nel bagaglio dell'esperienza."

Il programma non può basarsi su regole fisse (es. "se ha 4 zampe è un gatto", perché confonderebbe il cane; o "se ha la coda", perché confonderebbe pappagalli e pesci). Ha bisogno di:

1. **Esperienza:** Dati (es. immagini di gatti e non gatti, che devono essere etichettate).
2. **Misura di performance (P):** Quanto è bravo a classificare le foto. Meno errori commette, più è accurato.

Addestramento: Training-set e Test-set

Perché il classificatore deve apprendere? Per capire le caratteristiche del problema e adattarsi a condizioni diverse della realtà, dove la regola fissa non si può applicare. Se abbiamo **10.000 immagini**, il dataset si divide in:

- **Training-set:** Circa l'80% dei dati, serve per addestrare il classificatore e accumulare esperienza.

- **Test-set:** Il restante 20%, serve per valutare l'applicazione nella vita reale prima del rilascio.

In sintesi, la formula fondamentale è:

Modello = Algoritmo + Dati dell'esperienza (per fare previsioni)

La Matrice di Confusione

Per misurare l'accuratezza in una classificazione binaria ("è o non è", "gatto o non gatto") si usa la Matrice di Confusione, che mappa due possibili esiti veritieri e due errori:

- **Vero Positivo:** La macchina dice gatto $\Rightarrow$ ed è un gatto (Corretto).
- **Vero Negativo:** La macchina dice non gatto $\Rightarrow$ ed è un non gatto (Corretto).
- **Falso Positivo:** La macchina dice gatto $\Rightarrow$ ma è un non gatto (Errore).
- **Falso Negativo:** La macchina dice non gatto $\Rightarrow$ ma è un gatto (Errore).

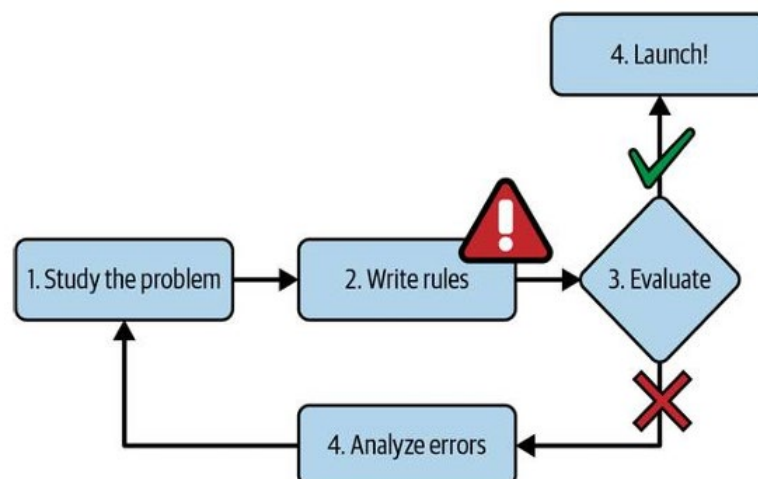
(Nota per l'esame: il classificatore ideale restituisce solo Veri Positivi e Veri Negativi. La presenza di Falsi Positivi e Falsi Negativi indica gli errori commessi).

Approccio Classico vs. Machine Learning

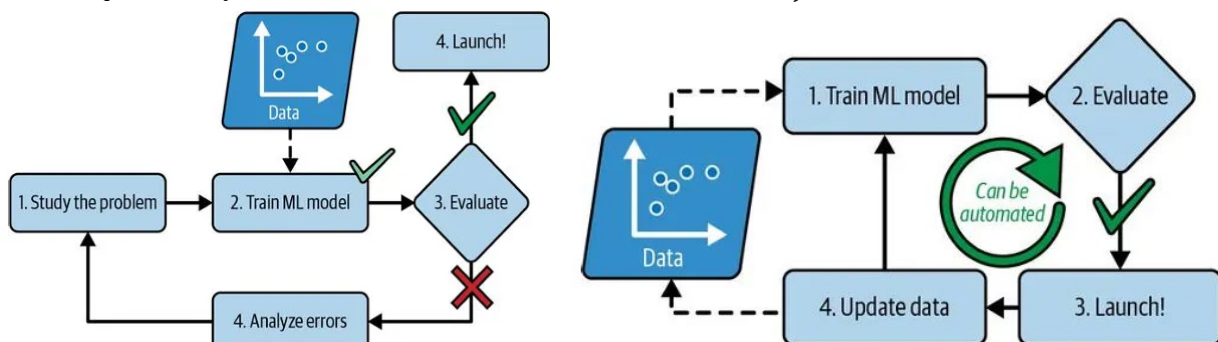
Perché usare il ML invece delle regole classiche? L'esempio storico è lo **spam filtering**.

Inizialmente usava condizioni if-else (es. se nel mittente, nell'oggetto o nel testo c'è la parola "win", allora è spam).

- **Approccio NO Machine Learning:** Studiare il problema $\rightarrow$ scrivere le regole $\rightarrow$ valutare. Se ci sono esiti favorevoli si lancia il software; se ci sono errori, bisogna analizzarli e tornare al punto 1 (riscrivere le regole a mano)



- **Approccio Machine Learning:** Studiare il problema $\rightarrow$ addestrare il modello $\rightarrow$ valutarlo. Se ci sono esiti favorevoli si lancia; se ci sono errori, si analizzano e si torna al punto 1 (il modello si riaddestra e si adatta da solo).



Conclusione: Un filtraggio basato su regole si basa su condizioni fisse. Un filtraggio basato sul ML si basa sull'esperienza, approfondisce gli schemi nascosti e **funziona meglio**. Il ML splende quando ci sono maggiori cambiamenti e in situazioni che richiedono adattamenti continui.

Dataset: Struttura e Visualizzazione

Un dataset (es. un file CSV) è una matrice strutturata così:

- **Istanze:** Le righe della matrice (i singoli casi).
- **Features:** Le colonne, ovvero gli attributi del dataset.
- **Target:** L'obiettivo da stimare/prevedere.

Il problema della visualizzazione (t-SNE): Se dobbiamo stimare il costo di abitazioni in un vicinato e abbiamo circa 10 colonne (features), è impossibile creare un grafico per grandi dimensioni. Bisogna ridurlo a massimo 2 o 3 dimensioni basandosi sugli attributi più importanti. Si usa il **visualizzatore t-SNE** che permette di visualizzare oggetti diversi tra loro. L'analisi grafica permette all'uomo di rilevare le anomalie, ma l'obiettivo finale è che sia il sistema (non l'uomo) a rilevarle automaticamente.

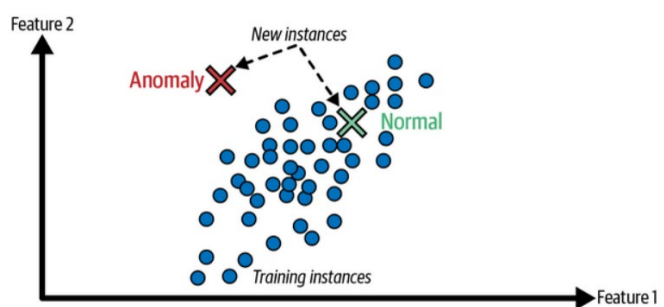
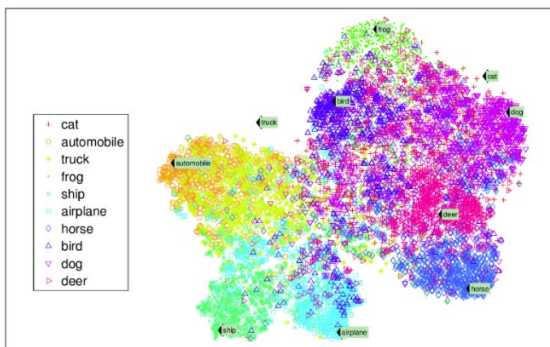


Figure 1-10. Anomaly detection

Famiglie di Machine Learning (in base al training set)

1. **Apprendimento Supervisionato:** I dati forniti hanno già le etichette.
 1. **Classificazione:** Può essere binaria (gatto/non gatto, spam/non spam) o non binaria (es. leggere cifre scritte a mano da una matrice 28x28 pixel).
 2. **Regressione:** *valori continui*. Si usa per prevedere un valore, come il costo di un'abitazione in base alla posizione geografica (vicino/lontano dal mare) o al guadagno dell'acquirente. La regressione permette di fare:
 - **Estrapolazione:** Prevedere un valore esterno ai dati massimi a disposizione. (*Esempio: ho i dati della probabilità di pioggia dal 1980 al 2025 → estrapolo la previsione sul 2026*).
 - **Interpolazione:** Prevedere un valore interno ai dati a disposizione, basandosi su ciò che c'è prima e dopo. (*Esempio: ho i dati dal 1980 al 2025 ma mi manca il 1983 → interpolo per trovare il 1983*).
2. **Apprendimento Non Supervisionato (Clusterizzazione):** I dati non hanno etichette. Il sistema suddivide (clusterizza) elementi, come le preferenze e gli acquisti degli utenti in un e-commerce, in gruppi senza supporto/supervisione umana. (Es. la *Customer segmentation*).

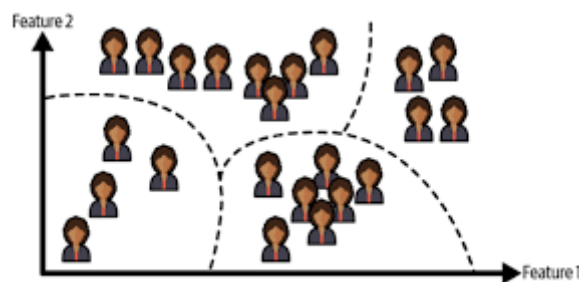


Figure 1-8. Clustering

3. **Apprendimento Semi-supervisionato**
4. **Auto-supervisionato**

5. Reinforcement Learning (Apprendimento per rinforzo):

Non è né supervisionato né non supervisionato. Un agente deve portare a termine un obiettivo (compenso di lungo termine). Ad ogni sua azione corrisponde un "premio" (per le mosse giuste) o una "penalità" (per gli errori). Tipico degli agenti che imparano a giocare.

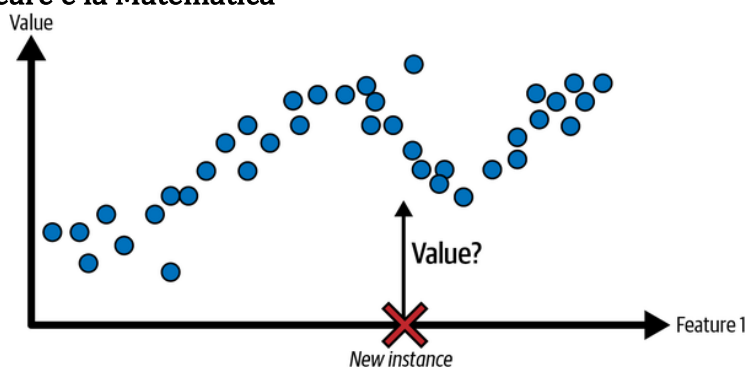
6. Ensemble Learning: Più modelli lavorano a catena: il primo esegue la classificazione; il secondo lavora solo sui dati in cui il primo ha fallito; dove fallisce il secondo entra in azione il terzo, e così via.

Altre Applicazioni del Machine Learning

Oltre a quelle già citate:

- Rilevazione di tumori tramite scansioni mediche.
- Etichettatura di immagini per prodotti specifici.
- Identificazione del *sentiment* (atteggiamento/emozione) in un testo.
- **TLDR (Too long, didn't read)**: Fornire riassunti automatici di testi lunghi.
- Previsione di guadagni (tramite modelli di regressione).
- Identificazione delle frodi finanziarie.

La Regressione Lineare e la Matematica



La regressione lineare NON è un algoritmo di classificazione, ma di apprendimento lineare. Ipotizziamo di avere dei dati con un trend di crescita (es. soddisfazione di vita rispetto al PIL pro capite). I dati non si disporranno mai perfettamente su una linea dritta, ma bisogna misurare la tendenza tramite gli errori.

La formula è:

$$life_satisfaction = \theta_0 + \theta_1 \cdot GDP_per_capite$$

Questa formula non è altro che l'equazione della retta $y = q + mx$, dove:

- *GDP_per_capite* (PIL pro capite) è la nostra x .
- θ_1 rappresenta la m (la pendenza della retta).
- θ_0 rappresenta la q (l'intercetta o ordinata all'origine).

Come si sceglie la retta migliore?

L'indice di qualità si basa sulla retta che si avvicina di più in assoluto ai dati sperimentali ("best fit"). Bisogna fare in modo che la **somma delle distanze (residui)** tra la retta e i dati reali sia la minore in assoluto.

Per farlo si usa l'**operazione di minimo** (calcolando la derivata parziale per ogni dato).

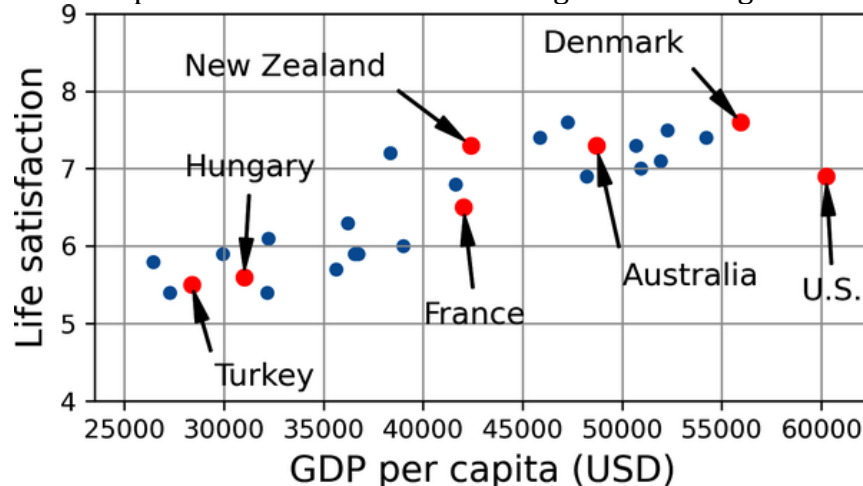
L'analogia matematica del prof: "È come nei problemi in cui cerchi il rettangolo inscritto in una circonferenza che abbia area massima, o il triangolo circoscritto con area minima → si calcolano ponendo a zero la derivata prima".

LEZIONE 4 – 16/03/2026 – CAP. 1 (GÉRON)

Recap: Il Machine Learning è la capacità di alcuni programmi di apprendere dai dati tramite l'esperienza.

La regressione lineare e la Matematica del modello

La regressione lineare è più antica del Machine Learning e dell'Intelligenza Artificiale.



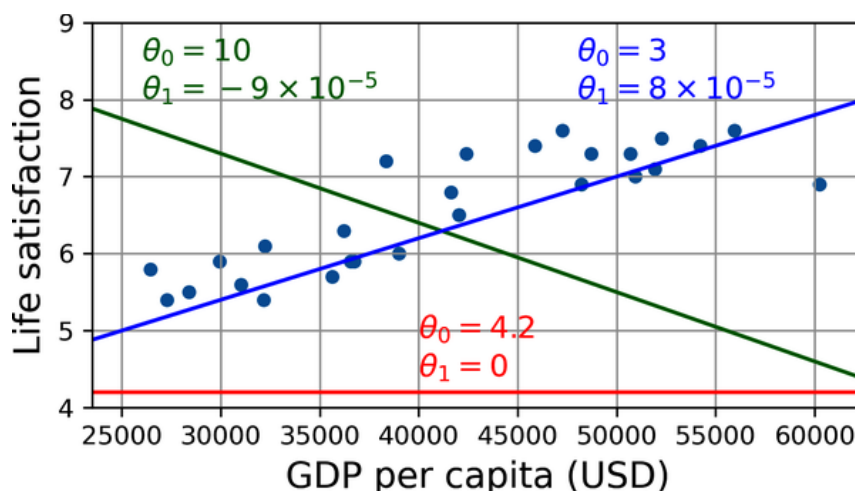
Se ci troviamo di fronte a un problema lineare, i dati possono essere rappresentati tramite una tendenza (trend). Ad esempio, mettendo in relazione il **GDP (PIL pro capite)** con la **soddisfazione di vita**: all'aumentare del GDP, aumenta anche la soddisfazione (comportamento linearmente crescente, ovvero una retta con pendenza positiva). Il nostro scopo è far imparare alla macchina questa relazione per poter fare delle stime future.

Anche se la matematica non si allinea mai perfettamente con la vita reale, la regressione lineare permette di ottenere la curva (in questo caso una retta) che **meglio identifica la tendenza generale**

$$\text{life.satisfaction} = \theta_0 + \theta_1 \cdot \text{GDP_per_capite}$$

Che corrisponde all'equazione classica della retta:

$$y = mx + q$$



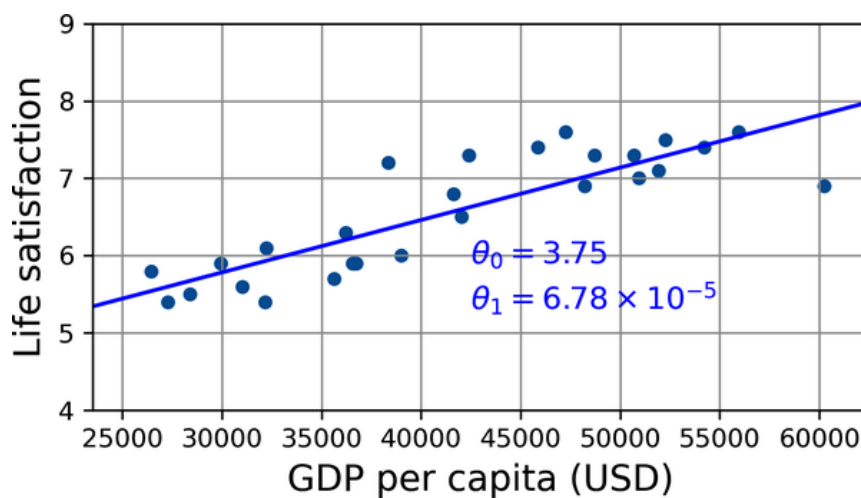
dalla quale:

- verde: insoddisfazione – ha una pendenza negativo $\Rightarrow$ più un paese è ricco, meno le persone sono soddisfatte
- rosso: lineare – è inutile, non segue i dati

- blu: segue l'andamento dei dati. Si tratta di una retta che segue il trend ascendente dei dati sperimentali: non può sovrapporsi a ogni punto, ma rappresenta l'algoritmo di best fit, vale a dire la tendenza generale migliore.

I parametri θ_0 e θ_1 vengono identificati attraverso un **problema di minimo**: si calcolano le distanze tra i punti sperimentali reali e la retta. Queste distanze rappresentano l'**errore**: minore è l'errore, migliore è l'approssimazione della retta.

Matematicamente, si calcola la derivata della funzione e la si pone uguale a zero. Essendo un problema a n variabili, allora si calcolano n derivate parziali (operazione automatica dai software). Tuttavia, questa è una soluzione di **compromesso**: forzare la retta a passare per un punto specifico, altera tutte le altre distanze.



Questa è una scala dilatata dell'asse x, altrimenti questa sarebbe una curva piatta a causa di 10^{-5}

Quindi la retta che ci dice la relazione tra GDP per capite e la soddisfazione di vita è:

$$y = 6.78 \times 10^{-5}x + 3.75$$

dove: $y = \text{life.satisfaction}$ e $x = \text{GDP_per_capite}$

In questo caso, fissando $y = 9$ (soddisfazione di vita molto alta), posso calcolare la x (GDP_per_capite)

$$y = mx + q \Rightarrow x = \frac{y - q}{m} = \frac{9 - 3.75}{6.78} \cdot 10000 \approx 78000$$

E se il problema NON è lineare?

Se i dati hanno un andamento **quadratico** $y = ax^2$, allora si può usare un trucco matematico (la sostituzione). Pongo $t = x^2$ e trasformo l'equazione di partenza in $y = at$. A questo punto, il problema è diventato lineare, quindi lo risolvo usando la normale regressione lineare. Dopo averlo risolto, ritrasformo tutto in quadratico.

Le **4 fasi di un progetto di Machine Learning** sono:

1. **Studiare i dati**: capirne la struttura e valutarne la qualità
2. **Selezionare il modello**: scegliere una o più famiglie di modelli adatti
3. **Addestrare il modello**: trovare i parametri perfetti che minimizzano la funzione di costo (l'errore)
4. **Fare previsioni** (Inferenza): usare il modello per stimare un valore continuo (Regressione) o per categorizzare i dati (Classificazione binaria o multiclasse).

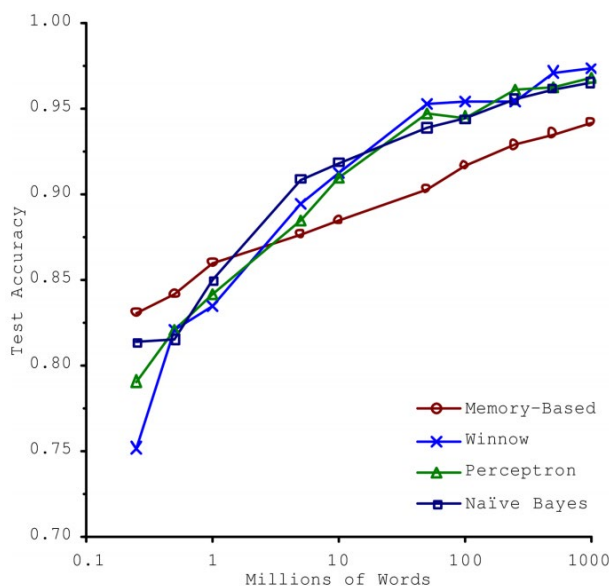
I Dati: Quantità, Costo e Rappresentatività

Se avessimo a disposizione **tutti** i dati possibili dell'universo (il GDP di tutti i paesi in ogni istante, o tutte le foto del mondo già etichettate), non ci servirebbe fare previsioni. Ma la realtà è diversa, è improbabile avere tutti i dati a disposizione.

Quantità e Costo Computazionale

Uno degli esempi principali è la classificazione delle cifre scritte a mano. Il dataset standard è composto da **70000 istanze** (immagini), della quale ognuna è una griglia di $28 \times 28 = 784$ pixel. Di conseguenza, il dataset è una matrice di dimensione 70000×785 (784 pixel + 1 colonna per l'etichetta). Il software impara distinguendo i pixel colorati: può arrivare al 99,9% di precisione ma mai al 100% perché il fattore umano inserisce delle imperfezioni (un 8 scritto male può sembrare un 9 o un 3).

Avere 1000 dati è meglio che averne 784, ma più dati si hanno più è maggiore il costo computazionale; quindi, il progettista deve trovare un **compromesso tra la quantità/qualità dei dati e il costo di addestramento**.



Il grafico risponde a una delle domande più antiche dell'IA: *è meglio avere un algoritmo intelligentissimo e super complesso, o avere una valanga di dati?*

- l'asse x rappresenta la **quantità di dati** usati per l'addestramento attraverso una scala logaritmica → training set
- l'asse y rappresenta le **prestazioni**, vale a dire quanto il modello è preciso sui dati diversi → test set
- le **4 linee colorate** rappresentano i **4 diversi algoritmi di classificazione del Machine Learning**

Analizzando l'andamento delle curve si può notare:

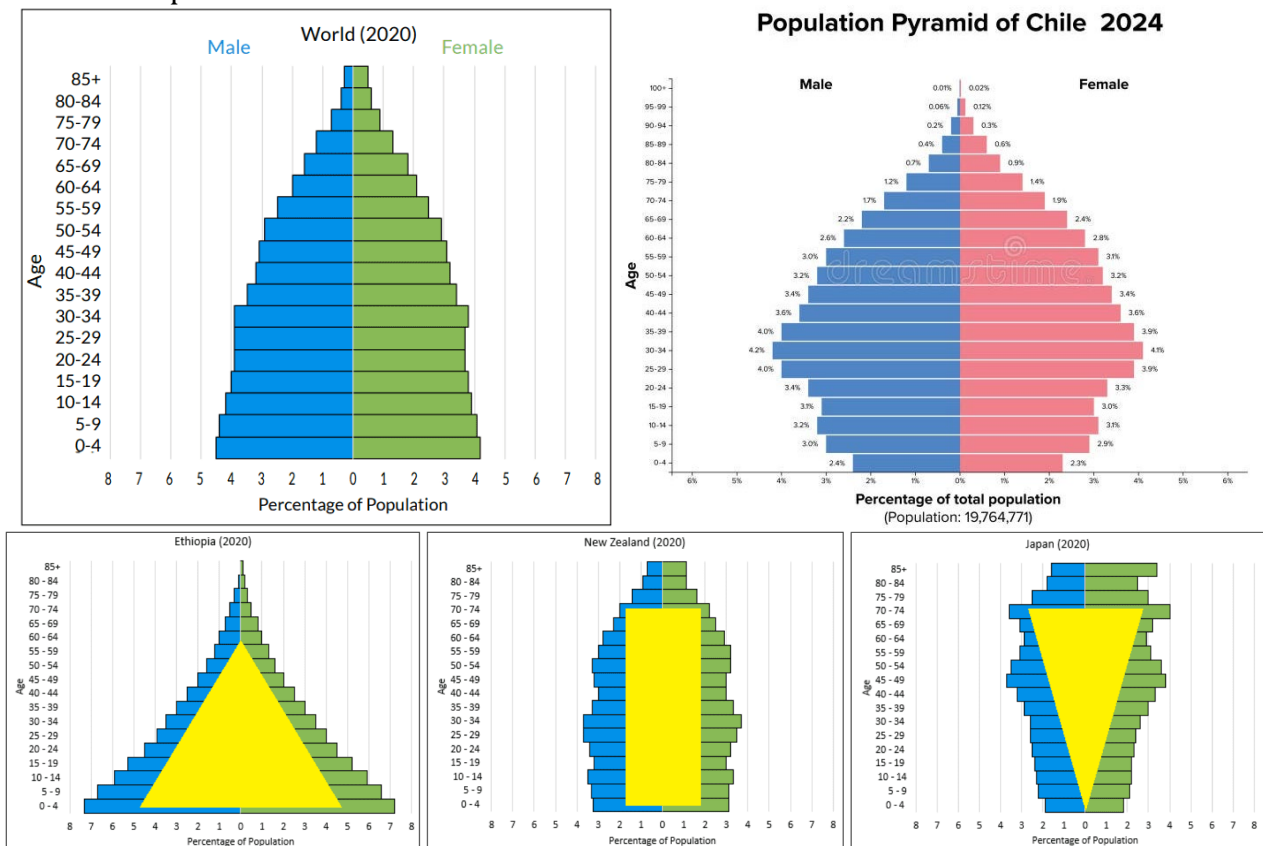
1. **scarsità di dati** (a sinistra): il modello Memory-Based è il migliore, mentre il Winnow è il peggiore
2. **abbondanza di dati** (a destra): con l'aumento dei dati, tutti gli algoritmi migliorano, in particolare il Winnow, che da peggiore diventa il migliore

Di conseguenza, avere più dati aumenta l'accuratezza ma bisogna sempre fare attenzione al costo computazionale.

Rappresentatività del campione: i dati non devono essere solo tanti, devono rappresentare il problema reale:

- **GDP per capite:** Se uso solo i dati dei paesi più ricchi o solo dei più poveri, il modello non rappresenterà la realtà globale
- **Foto:** Se addestro la macchina solo con foto di gatti, non saprà mai riconoscere cani o scoiattoli
- **Demografia:** l'età dei campioni deve rispecchiare l'obiettivo:
 - per studi alimentari serve la popolazione totale;
 - per studi sulla digitalizzazione ha senso usare un range (es. 6-50 anni).

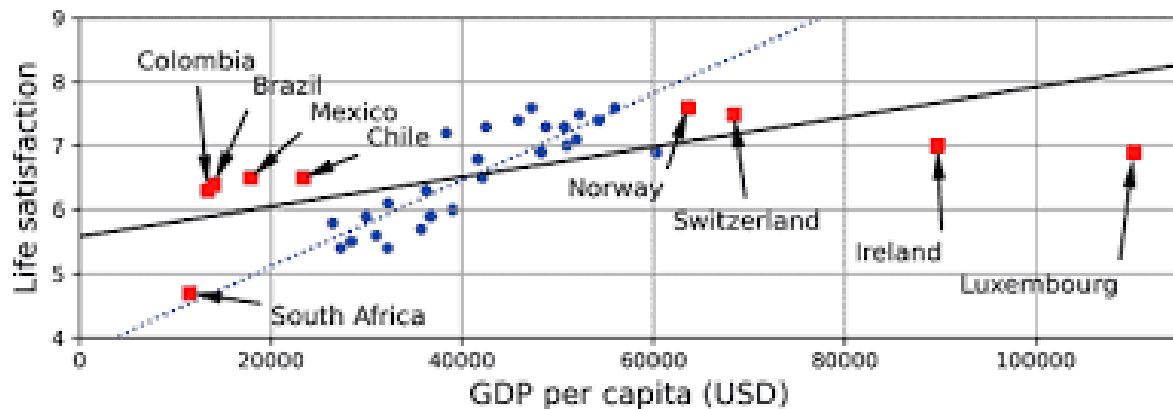
I paesi asiatici o africani hanno piramidi demografiche con molti giovani e pochi anziani, cosa che influisce profondamente sul dataset.



L'aneddoto di Gallup: l'istituto di sondaggi Gallup è famoso per aver sempre indovinato il Presidente degli Stati Uniti grazie all'abilità di selezionare un campione elettorale perfettamente rappresentativo dell'intera popolazione (election poll)

I limiti dei modelli e il Problema della Non-Linearità

Un altro grande problema nel Machine Learning è la **limitatezza dei modelli** che si scelgono di utilizzare. La regressione lineare si basa sulla forte assunzione che i dati seguano un andamento rettilineo. Ma cosa succede se proviamo a usare un modello **non lineare** (un polinomio di grado elevato) per cercare di essere più precisi sul dataset "GDP per capita vs Life Satisfaction"?



di cui:

- i quadrati **rossi a sinistra**: hanno un PIL pro capite piuttosto basso, ma una soddisfazione di vita sorprendentemente alta;
- i quadrati **rossi a destra**: hanno un PIL pro capite alto, ma la soddisfazione di vita è più bassa rispetto alle aspettative

La **retta tratteggiata blu** è addestrata solo sui dati parziali, ma è errata perché se facciamo previsioni sul Lussemburgo, ci sarebbe un errore e ciò succede perché il modello si è specializzato solo su un set di dati limitato e non sa gestire le eccezioni che ci sono nella realtà

La **retta nera continua** è addestrata su tutti i dati: è una retta di compromesso, passa meno in mezzo ai pallini blu, ma fa previsioni più realistiche per i paesi anomali come il Brasile o il Lussemburgo.

Overfitting vs. Underfitting (Il problema della generalizzazione)

Uno dei grandi limiti del Machine Learning (come la regressione lineare che assume rigidamente che tutto sia lineare) è la gestione dell'apprendimento.

La metafora degli studenti: ci sono 3 alunni a scuola:

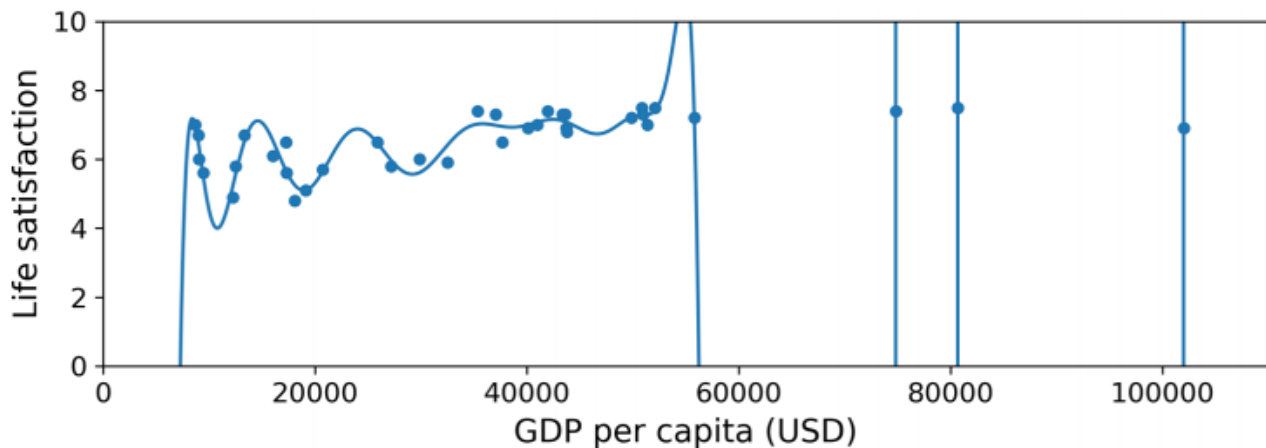
1. Il primo fa tutti gli esercizi a memoria senza capire.
2. Il secondo non ne fa nessuno.
3. Il terzo fa quelli richiesti ma *capisce la regola generale*.

Il giorno della verifica, il terzo prenderà il voto più alto. Il primo, di fronte a un problema diverso, andrà male.

Nella realtà questi tre casi si trovano nel mondo del machine learning a causa dell'overfitting e dell'underfitting.

Overfitting (Sovradattamento - *Lo studente 1*): il modello non capisce lo schema generale, ma memorizza i dettagli e i "rumori" dei dati di addestramento (training set). Si ha un'ottima corrispondenza nella fase di training, ma un errore altissimo sui nuovi dati nella fase di test.

- **Soluzione:** la **Regolarizzazione**. Si inseriscono dei vincoli matematici per evitare grandi oscillazioni della curva e mantenere i parametri piccoli e semplici.



Underfitting (Sottoadattamento - Lo studente 2): il modello ha prestazioni pessime. Si registrano errori molto alti sia in fase di training che in fase di testing.

- *Soluzione:* non è un problema di dati, ma si sta usando il modello sbagliato. Bisogna sceglierne uno più sofisticato.

L'Errore di Generalizzazione è proprio la misura dell'errore che il modello commette quando affronta nuovi dati (testing set) estratti dalla stessa distribuzione dei dati usati nel training.

Divisione del Dataset e Cross-Validation

Per valutare l'effettiva bontà di un modello (e scovare l'overfitting), il dataset viene suddiviso.

Suddivisione Standard:

- **Training set (80%):** per addestrare il modello, vale a dire per capire la pendenza e intercetta della retta
- **Testing set (20%):** per vedere quanto funziona correttamente la previsione su dati mai visti.

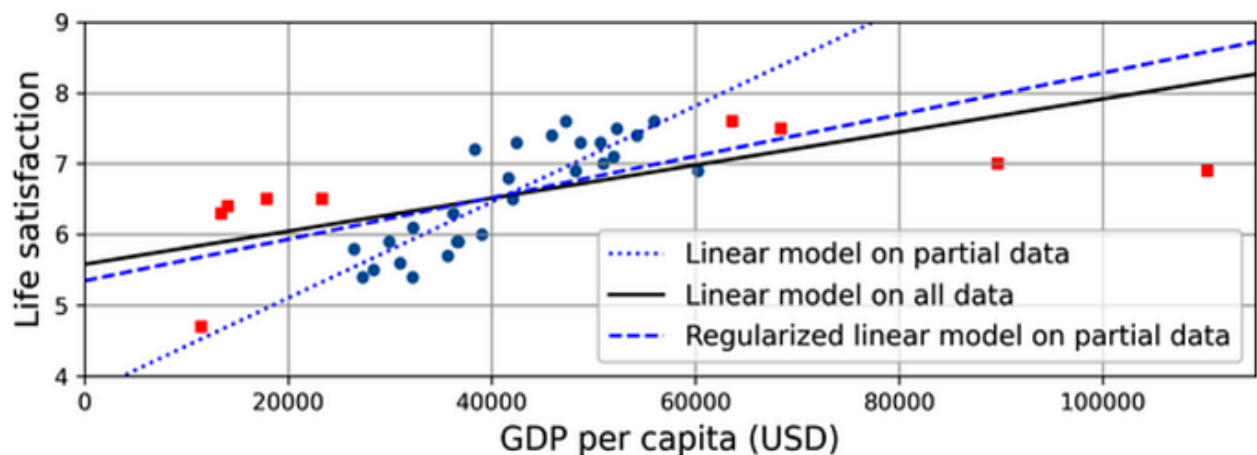
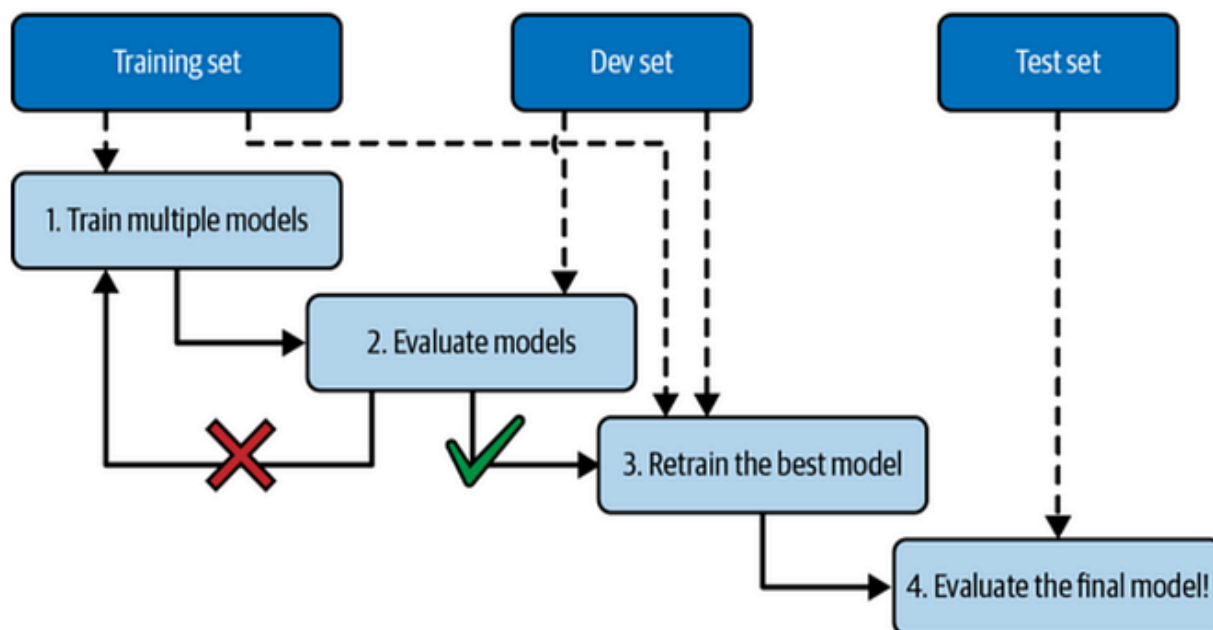


Figure 1-24. Regularization reduces the risk of overfitting

Suddivisione in Ambienti Industriali:

- **Training set (80%)**
- **Dev set / Validation set (10%):** usato in corso d'opera per valutare e mettere a punto le prestazioni di modelli diversi.
- **Test set (10%):** usato solo alla fine per la validazione assoluta del modello scelto.



La Tecnica della Cross-Validation (Convalida Incrociata): se abbiamo 70.000 immagini, le dividiamo in 7 “pacchetti” da 10.000. Si usano i primi 60.000 dati per il training e gli ultimi 10.000 per il test. Poi si “ruota”: si scelgono 60.000 dati diversi, cioè diversi pacchetti, per il training e gli altri 10.000 per il test. Alla fine, si ottengono 7 valori di precisione $P_1, \dots, P_7$. **La media di questi 7 valori dà la precisione finale del modello.** Questa tecnica è importante per evitare bias involontari (come l'esclusione di donne, anziani, o giovani), garantendo che il modello si alleni su diverse combinazioni, anche se comporta un notevole aumento del costo computazionale.

Procedura Tipica e Conclusioni

Il Machine Learning (che si divide in supervisionato, non supervisionato e semi-supervisionato) è estremamente più flessibile rispetto all'implementazione manuale di regole rigide. Tuttavia, il vero lavoro prima ancora di far girare gli algoritmi è la **collezione e pulizia dei dati**.

- Pulire i dati (eliminare righe/colonne vuote, corrotte o inutili) porta alla risoluzione del problema in tempi molto minori.
- È fondamentale un'attenta **selezione delle feature (colonne)**.

Il successo di una IA non dipende solo dall'algoritmo, ma dipende primariamente dalla qualità dei dati, dalla corretta scelta delle colonne/attributi, dalla scelta della famiglia dei modelli e da una rigorosa valutazione finale.

LEZIONE 5 – 17/03/2026 – CAP. 1 (GÉRON)

La classificazione

La **classificazione** è la seconda grande famiglia di metodi del **Machine Learning Supervisionato**. A differenza della regressione, che stima valori continui, la classificazione serve a suddividere i dati in categorie specifiche (distinzione tra foto di cani e foto di gatti).

I Dataset Storici (Le "Baseline")

Le *baseline* sono dei dataset standard e famosissimi usati dai programmatori per testare e confrontare gli algoritmi.

L'“Hello World” del Machine Learning è l'**Iris Dataset**: è stato introdotto nel 1936 da Ronald Fisher. Il dataset è piccolo ed è composto da:

- **150 istanze (campioni)**: sono stati analizzati 150 fiori di Iris in totale
- **3 classi (Target)**: i fiori sono divisi in 3 specie diverse, che rappresentano l'obiettivo della classificazione:
 - Iris **Setosa**: caratterizzata da dimensioni relativamente piccole.
 - Iris **Versicolor**: di dimensioni moderate, con caratteristiche a metà strada tra le altre due.
 - Iris **Virginica**: generalmente la più grande tra le tre specie.
- **4 Features (Caratteristiche)**: per ogni singolo fiore sono state misurate 4 variabili numeriche (in centimetri):
 - **Lunghezza del sepalo** (Sepal Length)
 - **Larghezza del sepalo** (Sepal Width)
 - **Lunghezza del petalo** (Petal Length)
 - **Larghezza del petalo** (Petal Width)

Questo dataset è importante nel Machine Learning per:

- la semplicità: avendo solo 4 attributi
- benchmark: viene usato per testare e confrontare la precisione di algoritmi diversi
- studio delle features grazie al set limitato dei dati

In Python, il dataset è integrato nativamente nella libreria `scikit-learn` più precisamente nella funzione `load_iris()` del modulo `sklearn.datasets`

Il MNIST Dataset (Cifre scritte a mano) è un dataset molto più complesso composto da **70.000 immagini** di cifre da 0 a 9. Ogni immagine è una griglia di **28x28 pixel** (totale 784 pixel). Le immagini sono in scala di grigi: ogni pixel ha un valore numerico che va da **0** (nero) a **255** (bianco). Il **software** non ha gli occhi per vedere la "forma" del numero, ma analizza la sequenza di questi 784 numeri per trovare degli schemi ricorrenti e stabilire la cifra rappresentata.

Curiosità storica: originalmente, i Babilonesi scrivevano il numero 3 con tre linee orizzontali e il 2 con due linee. Gli unici numeri che sono rimasti visivamente “fedeli” alla loro logica originaria sono lo 0 e l'1.

Strumenti Software e Cultura Pop

Scikit-learn (sklearn) è la libreria (modulo) di Python per eccellenza per il Machine Learning: contiene algoritmi pronti all'uso e generatori di dataset di prova (permette di generare nuvole di punti artificiali con un certo livello di imprecisione/rumore per testare se l'algoritmo riesce a separarle in classi).

Il magico numero **42**: quando si programmano questi algoritmi, spesso bisogna generare dei numeri casuali impostando un "seme di partenza" (random seed). Si usa quasi universalmente il numero 42. È una citazione alla saga di libri “Guida galattica per gli autostoppisti”, in cui un super-calcolatore, dopo milioni di anni di calcoli, stabilisce che 42 è la risposta alla domanda fondamentale sulla vita, l'universo e tutto quanto.

Rappresentatività dei Dati e Campionamento Stratificato

Uno dei parametri cruciali per la qualità dei dati è la loro **rappresentatività**. Quando si divide un dataset (per applicare la cross-validation), bisogna assicurarsi che le proporzioni originali vengano mantenute. Se nel dataset generale ci sono il 10% di numeri “5”, anche nei piccoli sottoinsiemi di training e test ci deve essere circa il 10% di numeri “5”. Questo processo si chiama **campionamento stratificato** (eseguibile tramite le funzioni di *Scikit-learn*).

Matrice di Confusione e Metriche di Qualità

La **matrice di confusione** è costituita da 4 possibili risultati che vengono prodotti dalla macchina:

- **Vero Positivo (TP - True Positive):**
 - l'immagine *è un 5*, e la macchina dice “Sì, è un 5”.
 - risultato: successo!
 - un mancato TP diventa un FN
- **Vero Negativo (TN - True Negative):**
 - l'immagine *NON è un 5*, e la macchina dice “No, non è un 5”
 - risultato: successo!
 - un mancato TN diventa un FP
- **Falso Positivo (FP - False Positive):**
 - l'immagine *NON è un 5*, ma la macchina dice “Sì, è un 5”
 - risultato: falso allarme
- **Falso Negativo (FN - False Negative):**
 - l'immagine *è un 5*, ma la macchina dice “No, non è un 5”
 - risultato: svista o un mancato rilevamento

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

Dalla matrice di confusione si misura la qualità dei dati tramite delle metriche: accuratezza, precisione e sensibilità.

L'**accuratezza** misura quante volte la macchina ha indovinato in totale ed è data dal rapporto tra le previsioni corrette totali e il totale delle previsioni effettuate:

$$\text{accuratezza} = \frac{\text{numero di previsioni corrette}}{\text{numero totale di previsioni}} = \frac{TP + TN}{TP + TN + FP + FN}$$

- se il classificatore è perfetto (solo TP e TN) allora l'accuratezza è 1 (100%)
- se il classificatore sbaglia tutto (solo FP e FN) allora l'accuratezza è 0

La **precisione** stabilisce quante volte la macchina indovina realmente:

$$P = \frac{TP}{TP + FP}$$

Il **recall** o **sensibilità** si concentra sulla capacità di non farsi sfuggire nulla, punendo le sviste:

$$R = \frac{TP}{TP + FN}$$

A volte, l'accuratezza è ingannevole e i dati sono sbilanciati; di conseguenza si usa l'F1-Score che è la media armonica tra precisione e recall e serve a trovare un equilibrio tra essi:

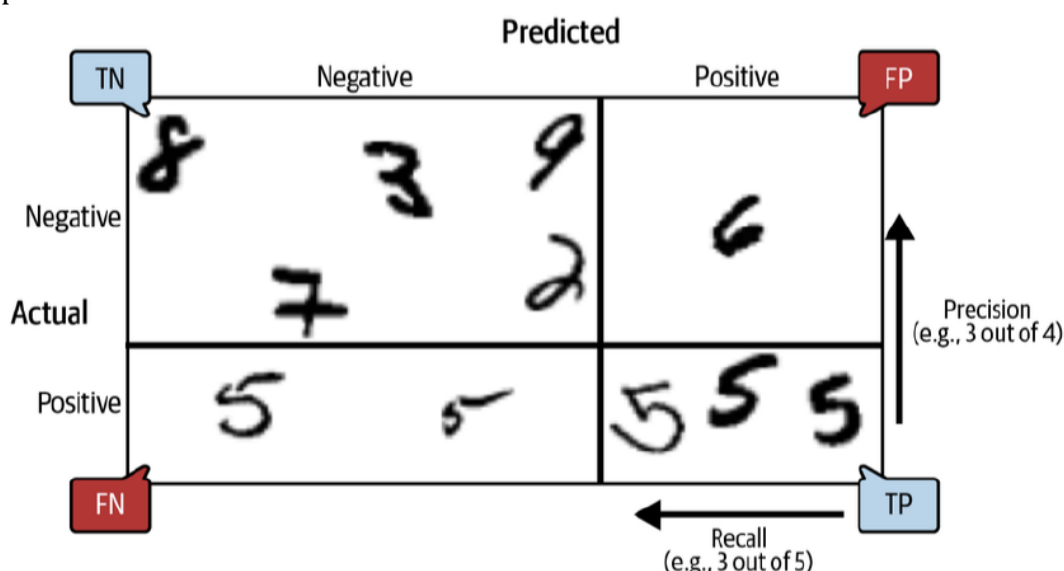
$$F1 = 2 \cdot \frac{P \cdot R}{P + R}$$

- se $P = R = 1$ allora l'F1-Score è perfetto, cioè 1

LEZIONE 6 – 23/03/2026 – CAP. 3 (GÉRON)

Lavoreremo sul dataset MNIST (70.000 immagini di cifre scritte a mano). Immaginiamo di creare un classificatore binario: la macchina deve solo dirci "È un 5" (Positivo) o "Non è un 5" (Negativo).

Perché l'**Accuratezza** (previsioni corrette / totale) non basta? Se le cifre "5" sono solo circa il 10% del totale (quindi il 90% dei dati *non* sono "5"), un modello "stupido" che risponde sempre "Non è un 5" senza nemmeno guardare l'immagine avrà un'accuratezza del 90%! Ecco perché usiamo la **Matrice di Confusione**.



Precisione (Precision): Guarda solo la colonna delle previsioni positive (2°). Delle volte che il modello ha detto "È un 5", quante volte aveva ragione?

$$P = \frac{TP}{TP + FP}$$

Idealmente vorremmo zero Falsi Positivi ($FP = 0$), ottenendo una precisione di 1 (100%).

Recall (Sensibilità): Guarda la riga dei valori veri positivi (2°). Di tutti i veri "5" che c'erano, quanti ne ha trovati il modello?

$$R = \frac{TP}{TP + FN}$$

F1-Score: È la media armonica tra Precisione e Recall. Premia i modelli che hanno valori simili per entrambe le metriche.

$$F1 = 2 \cdot \frac{P \cdot R}{P + R}$$

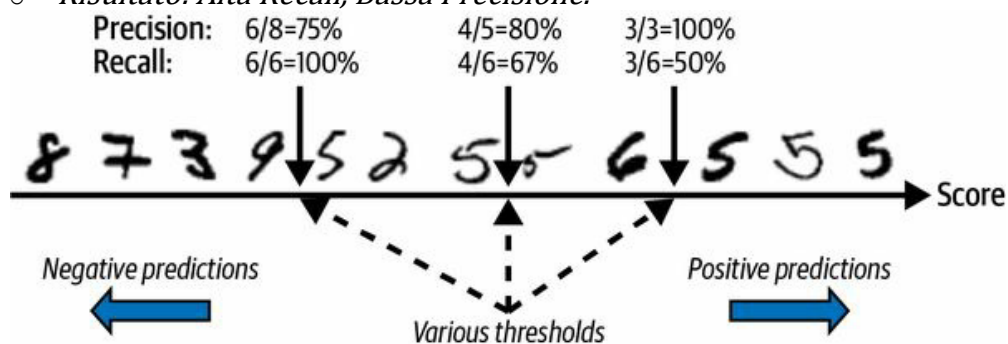
Se pongo $P = 0.5$ e $R = 1$ allora $F1 = 0.6$

Ridurre i FN (Falsi Negativi → le sviste), **migliora** la Recall. Infatti, dalla formula $R = \frac{TP}{TP+FN}$, più FN si avvicina a 0, il denominatore diventa più piccolo e il risultato si avvicina a 1 (il massimo).

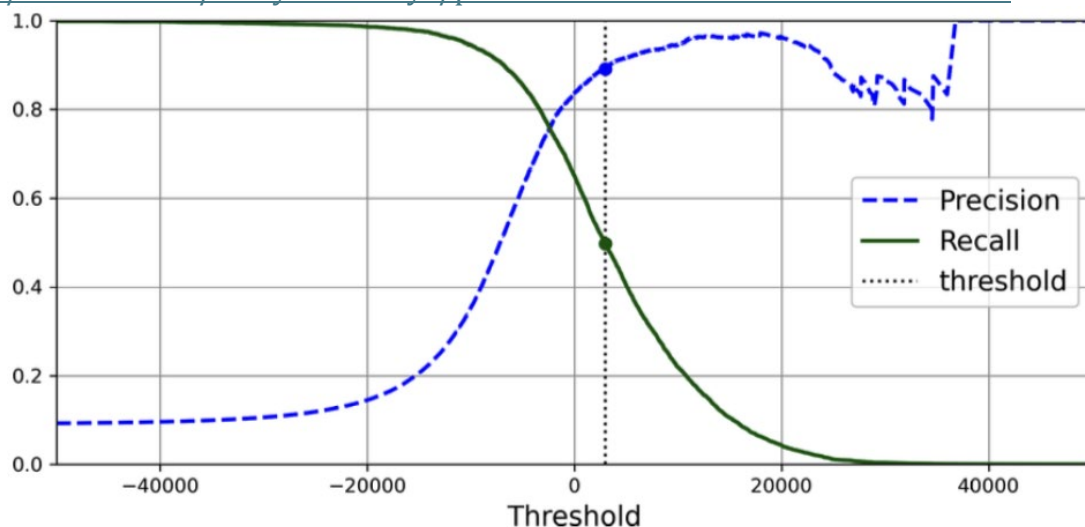
Tuttavia, c'è un **trade-off (compromesso)**: se si aumenta la precisione (P) allora si abbassa il recall (R) e viceversa.

esempi:

- CASO A: video per bambini (ALTA PRECISIONE)
Il modello deve bloccare i contenuti violenti. Non vogliamo **nessun Falso Positivo** (cioè un video violento classificato per sbaglio come "sicuro"). Per assicurarci di questo, il modello diventerà iperprotettivo e scarterà anche dei cartoni animati innocui (aumentando i Falsi Negativi).
○ Risultato: Alta Precisione, Bassa Recall.
- Caso B: I Ladri nel negozio (ALTA RECALL)
Immagina le telecamere di sicurezza che devono segnalare un taccheggiatore. Il nostro obiettivo è non far scappare nessun ladro. Non vogliamo nessun **Falso Negativo**. Per essere sicura di prenderli tutti, la telecamera suonerà anche quando un cliente innocente si mette le mani in tasca per prendere il cellulare (generando molti Falsi Positivi).
○ Risultato: Alta Recall, Bassa Precisione.

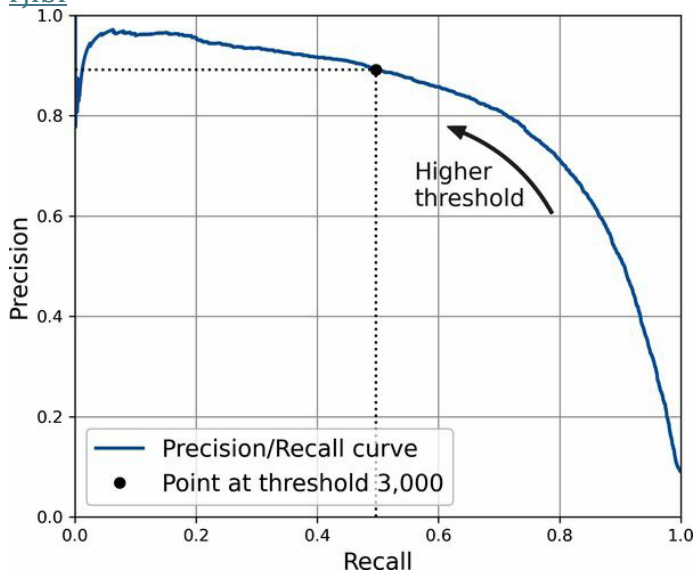


<https://medium.com/analytics-vidhya/precision-recall-tradeoff-79e892d43134>



le due curve non sono in concordanza, se aumenta una, l'altra diminuisce (precisione e recall sono inversamente proporzionali)

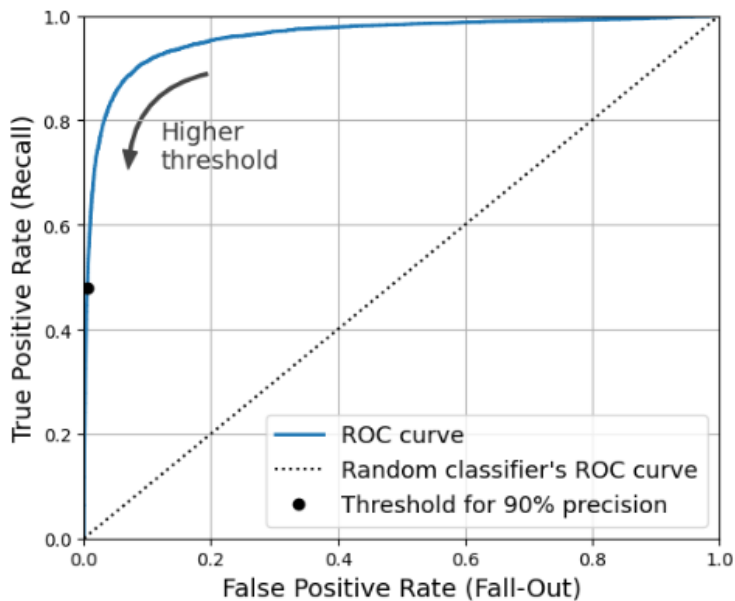
<https://www.linkedin.com/pulse/precision-recall-trade-off-finding-balancing-point-6-javier-rjibf>



La Curva ROC (Receiver Operating Characteristic)

La curva ROC mette a confronto il **True Positive Rate (Recall)** contro il **False Positive Rate (FPR)**.

FPR: è la percentuale di veri negativi che sono stati classificati erroneamente come positivi (i falsi allarmi)



Condizione peggiore?

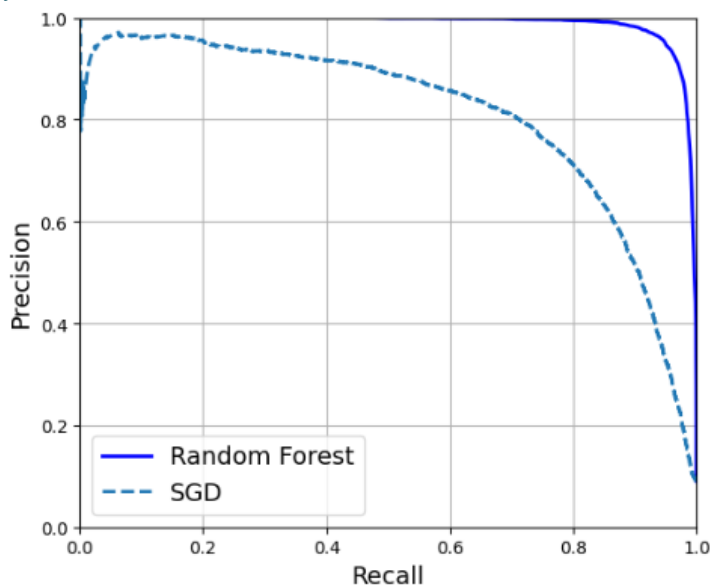
La retta diagonale tratteggiata. Rappresenta un classificatore che tira a indovinare (Random Classifier). Un modello del genere ha un'Area Under the Curve (AUC) pari a 0.5.

Condizione migliore?

Il classificatore ideale è quello che si spinge il più possibile verso l'angolo **in alto a sinistra** del grafico. Avrebbe un'AUC pari a 1.0 (il 100% dell'area del quadrato).

Un altro classificatore è `RANDOMFORESTCLASSIFIER`

<https://www.linkedin.com/pulse/beyond-sgdclassifier-comparing-pr-curve-f1-score-part-javier-l5uif>



Se le classi sono molto sbilanciate (come i 5 che sono solo il 10%), la curva PR è molto più onesta e utile della curva ROC per valutare il modello. Ad esempio, il modello *RandomForestClassifier* produce curve PR e ROC decisamente migliori rispetto al modello lineare *SGD*.

Classificazione Multiclasse (Andare oltre il binario)

Cosa succede se vogliamo classificare tutte e 10 le cifre (da 0 a 9) e non solo i 5? Alcuni algoritmi lo fanno nativamente (Random Forest), altri (Support Vector Machines) sanno fare solo il binario.

Per usare un classificatore binario su più classi, ci sono due strategie:

- **One-vs-Rest (OvR):** Addestriamo 10 classificatori separati (“Zero vs Tutti”, “Uno vs Tutti”, ecc.) e prendiamo quello con il punteggio più alto. Ha un costo computazionale lineare $O(n)$.
- **One-vs-One (OvO):** Addestriamo un classificatore per *ogni singola coppia* possibile (“0 vs 1”, “0 vs 2”, “1 vs 2”, ecc.). Usa la formula delle strette di mano: $\frac{N(N-1)}{2}$. Ha un costo quadratico $O\left(\frac{n(n-1)}{2}\right) \approx O(n^2)$, ma ogni modello si allena solo su due cifre alla volta.

Scikit-Learn usa spesso OvO di default per algoritmi come SVM.

Analisi degli Errori e Feature Engineering

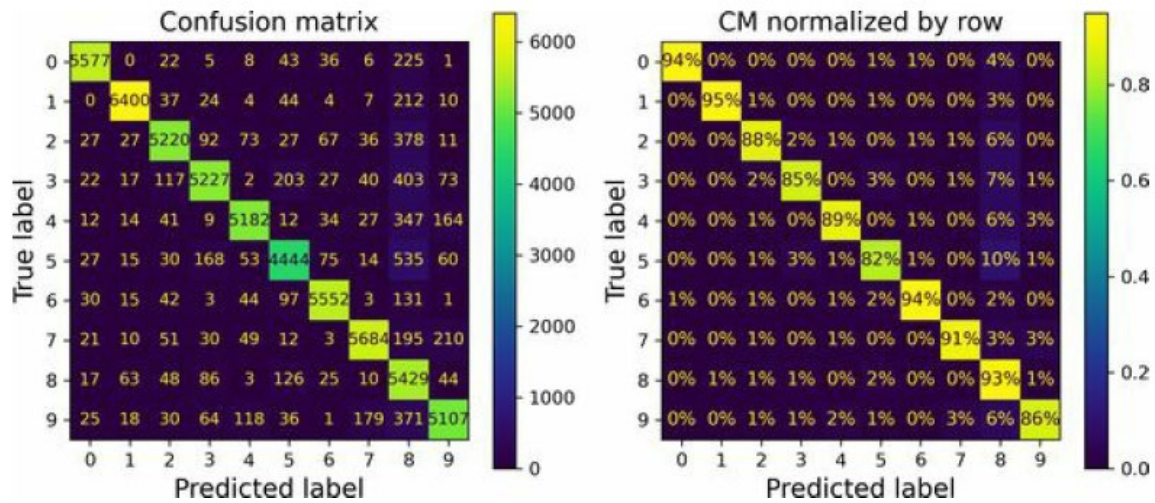
Matrici Normalizzate: se lavoriamo con 10 classi, avremo una matrice di confusione 10x10. Normalizzarla per riga ci permette di vedere le percentuali invece dei numeri assoluti.

Il trucco della diagonale: per capire dove il modello sbaglia, i programmatori spesso azzerano la diagonale principale (che contiene i successi) e lasciano solo le celle con gli errori. Così, guardando i colori accesi, capisci subito se il modello confonde spesso, ad esempio, i 3 con i 5.

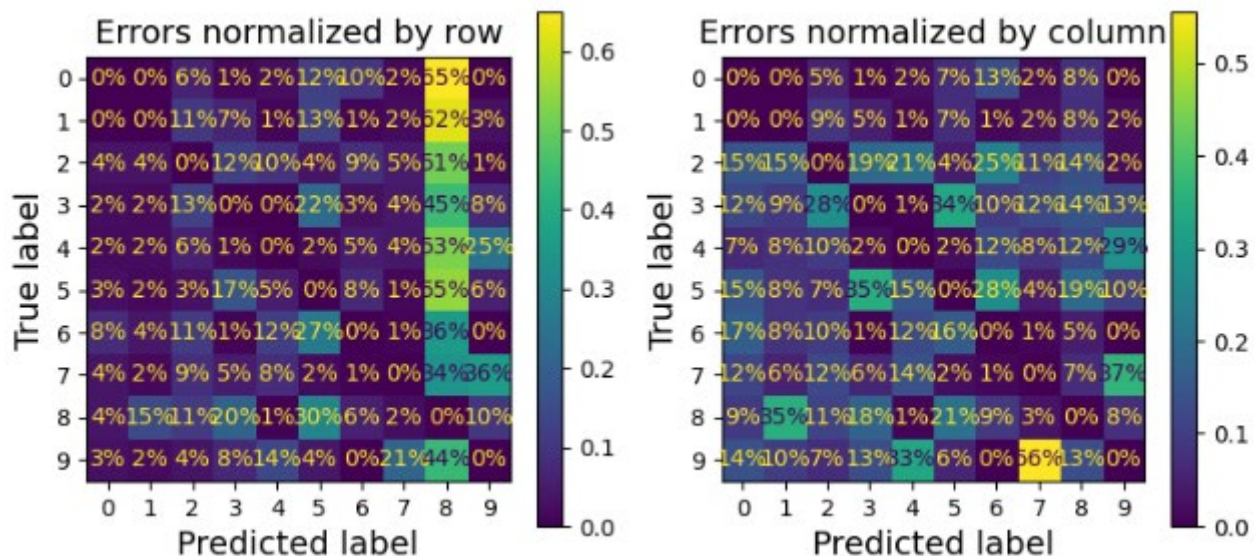
Feature Engineering: a volte la macchina non impara bene perché i dati grezzi non bastano. Bisogna “ingegnerizzare” nuovi attributi.

Esempio: invece di dare alla macchina solo “Numero di stanze (3)” e “Metri quadri (100)”, calcoliamo noi il rapporto (Stanze/Mq). Questo nuovo numero darà all'algoritmo un'idea immediata del “tenore” della casa.

<https://www.linkedin.com/pulse/delving-deeper-error-analysis-action-part-10-javier-monasterio-solar-hl8df>



<https://www.linkedin.com/pulse/delving-deeper-error-analysis-action-part-10-javier-monasterio-solar-hl8df>



Multilabel, Multioutput e Decision Trees

Multilabel: non è mutuamente esclusivo. Un'istanza può avere più etichette contemporaneamente. Restituisce un vettore.

esempi:

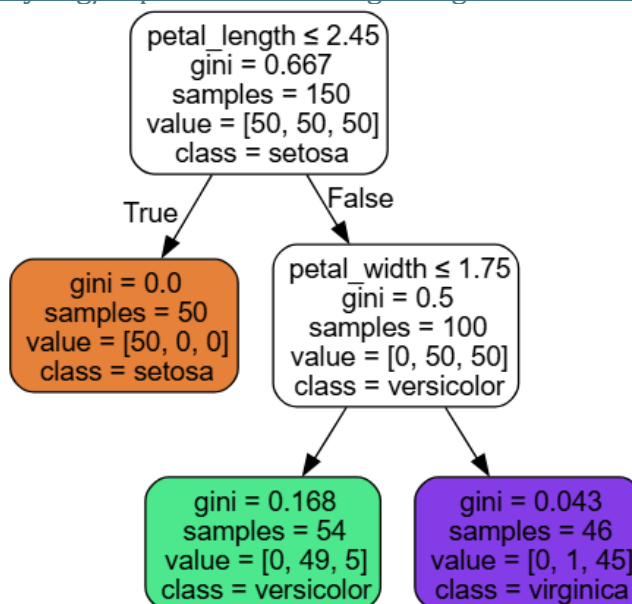
- La foto di un "7" può restituire un vettore di verità [True, True] se gli chiediamo "È un numero grande (≥ 7)?" e "È dispari?"
- foto che contiene più animali e/o persone

Multioutput: l'output non è una semplice etichetta, ma qualcosa di più complesso, come un'intera immagine! Un esempio è il *denoising*: l'algoritmo (es. KNN) riceve in input un'immagine piena di rumore e restituisce in output i valori puliti per ognuno dei 784 pixel.

Alberi Decisionali (Decision Trees): modello supervisionato molto intuitivo basato su decisioni a cascata (if/else). Il professore ha usato il famoso **Iris Dataset** (150 fiori, 3 varietà: *versicolor*, *setosa*, *virginica*). La divisione dei rami cerca di minimizzare l'impurità di **Gini** (più è vicina a 0, più il gruppo di dati in quel ramo è "puro" e appartiene a una sola classe).

Attenzione: se lasciamo crescere l'albero troppo in profondità senza limiti (classificazione puntuale su ogni singolo fiore), finiamo dritti nell'**Overfitting**.

<https://gamedevacademy.org/supervised-learning-using-decision-trees-to-classify-data/>



LEZIONE 7 – 30/03/2026 –

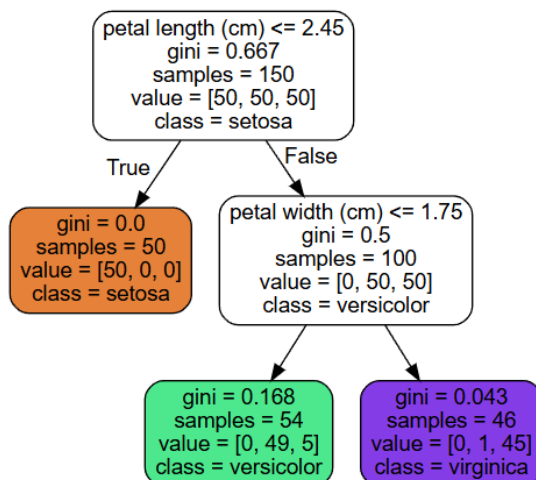
Gli alberi di decisione (Decision Trees)

Gli Alberi di Decisione sono algoritmi di Machine Learning (Apprendimento Supervisionato) estremamente intuitivi. A differenza di altri modelli, non cercano di tracciare equazioni matematiche complesse, ma si basano su una serie di **domande a cascata** (se/allora - *if/else*).

Il Funzionamento Base (L'esempio Iris)

Usando il famoso dataset Iris (150 fiori divisi in 3 specie: *Setosa*, *Versicolor*, *Virginica*), il modello impara a classificare usando solo 2 attributi (features): lunghezza e larghezza del petalo.

(Nota: Spesso i dataset hanno migliaia di attributi, ma l'albero di decisione ignora quelli inutili e seleziona solo quelli davvero determinanti).



Come ragiona l'albero:

1. **Nodo radice (Root):** L'algoritmo pone una prima soglia ottimale. Si chiede: *La lunghezza del petalo è ≤ 2.45 cm?*

- **Ramo VERO (Sinistra):** Ci finiscono 50 fiori. Sono TUTTI della classe *Setosa* [50, 0, 0]. Questo nodo non fa più domande, è puro. Si chiama **Foglia (Leaf)**.
- **Ramo FALSO (Destra):** Ci finiscono gli altri 100 fiori [0, 50, 50]. Il nodo è "impuro", quindi deve fare un'altra domanda.

2. **Nodo interno:** Si chiede: *La larghezza del petalo è ≤ 1.75 cm?*

- **Ramo VERO:** 54 fiori, di cui 49 *Versicolor* [0, 49, 5].
- **Ramo FALSO:** 46 fiori, di cui 45 *Virginica* [0, 1, 45].

Quando arriviamo a una foglia, il modello calcola la **probabilità** dividendo le istanze di una classe per il totale del nodo (es. nel ramo Versicolor: $\frac{49}{54} \approx 90.7\%$). La probabilità maggiore decide la classe finale predetta.

Le Metriche di Impurità

Come fa l'algoritmo a decidere qual è la domanda migliore da fare a ogni nodo? Cerca la domanda che divide i dati nei gruppi più "puri" possibili. Per misurare la purezza si usano due metriche:

1. L'Indice di Impurità di Gini

L'indice di Gini misura quanto è "misto" un nodo. Un nodo perfettamente puro (es. [50, 0, 0]) ha un'impurità pari a **0**. Più il nodo è misto, più l'indice sale.

$$G_i = 1 - \sum_{k=1}^n p_{i,k}^2$$

Dove $p_{i,k}$ è la proporzione (percentuale) della classe k all'interno del nodo i .

Esempio pratico di calcolo (Il nodo Versicolor [0, 49, 5]):

1. Totale fiori nel nodo: $0 + 49 + 5 = 54$.
2. Calcolo le proporzioni:
 - Classe 1 (Setosa): $p_1 = \frac{0}{54} = 0$
 - Classe 2 (Versicolor): $p_2 = \frac{49}{54} \approx 0.907$
 - Classe 3 (Virginica): $p_3 = \frac{5}{54} \approx 0.093$

3. Applico la formula:

$$G_i = 1 - (0^2 + 0.907^2 + 0.093^2)$$

$$G_i = 1 - (0 + 0.823 + 0.008) = 1 - 0.831 = 0.169$$

2. L'Entropia di Shannon

L'entropia è un concetto preso dalla fisica (es. non puoi trasformare tutto il lavoro in calore utile, pensa ai freni della bici che si scaldano per dissipare energia). Nella teoria dell'informazione, misura il disordine e l'imprevedibilità di un nodo.

$$H_i = - \sum_{k=1}^n p_{i,k} \cdot \log_2 p_{i,k}$$

Se un nodo è puro al 100% ($p = 1$), il $\log_2(1)$ è zero, quindi anche l'entropia del nodo puro è **0**.

Algoritmo CART e Funzione di Costo

Per addestrare l'albero si usa l'algoritmo **CART** (Classification and Regression Trees).

Cerca la coppia (*caratteristica k , soglia t_k*) che produce la suddivisione più pura calcolando questa **Funzione di Costo da minimizzare**:

$$J(k, t_k) = \frac{m_{left}}{m} \cdot G_{left} + \frac{m_{right}}{m} \cdot G_{right}$$

Una media pesata dell'impurità dei due nodi figli

- **Approccio Greedy:** CART è un algoritmo *Greedy* (avido), proprio come l'algoritmo di Dijkstra per i percorsi minimi o il problema dello zaino. Prende la decisione che sembra migliore *in quel momento specifico (locale)*, senza curarsi se una scelta diversa porterebbe a un albero **globale** migliore in futuro.
- **Complessità NP-Completa:** Trovare l'albero *perfetto* assoluto è un problema NP-Completo (costo computazionale esponenziale). Quindi ci accontentiamo dell'ottimo locale trovato dall'approccio Greedy.
- **Uso nella Regressione:** CART si usa anche per prevedere numeri continui. In quel caso, invece di minimizzare l'impurità di Gini, minimizza l'errore (la distanza al quadrato), esattamente come fa la Regressione Lineare per trovare la retta migliore in una nuvola di punti.

Dijkstra condivide la stessa "psicologia" dell'algoritmo **CART** (quello che costruisce gli Alberi di Decisione). Sono entrambi algoritmi Greedy!

- **Dijkstra:** "Qual è la città inesplorata più vicina a me *adesso*? Ci vado!"
- **CART:** "Qual è il singolo taglio dei dati (es. *Lunghezza petalo* < 2.45) che riduce di più l'impurità di Gini *adesso* in questo specifico nodo? Lo faccio!"

La differenza fondamentale: l'approccio Greedy di **Dijkstra** ha una particolarità magica: alla fine del processo, garantisce SEMPRE di aver trovato l'**ottimo globale** (il percorso realmente più corto in assoluto), a patto che non ci siano strade con chilometri negativi. L'approccio Greedy di **CART**, invece, **NON garantisce l'ottimo globale**.

Costo Computazionale vs Tempo di Esecuzione

Tempo di esecuzione: dipende dall'hardware (es. far girare il codice su un vecchio 386 ci metterà molto più tempo rispetto a un PC moderno).

Costo Computazionale: è il numero di operazioni matematiche necessarie, a prescindere dal PC:

- **Previsione (Inferenza):** velocissima. Basta scendere lungo l'albero. Costo: $O(\log_2(m))$.
- **Addestramento (Training):** Più lento. L'algoritmo deve confrontare tutte le feature (n) su tutte le istanze (m). Costo: $O(n \times m \log_2(m))$.

xAI, Interpretabilità e "Inverni dell'IA"

Gli Alberi di Decisione fanno parte della **xAI (Explainable AI)**.

- **White Box (Scatola Trasparente):** l'analista capisce perfettamente perché il modello ha preso una decisione (basta leggere gli if/else).
- **Black Box (Scatola Nera):** algoritmi come le Reti Neurali. Funzionano benissimo ma la logica matematica interna è incomprensibile all'uomo.

Gli Inverni dell'IA: nella storia dell'IA ci sono stati periodi di crisi (inverni) dovuti alla limitatezza delle risorse. Ci si rese conto che problemi complessi non potevano essere risolti solo con gli if. Ad esempio, per creare un'IA che vinca a scacchi, un gigantesco albero di decisione non basta, servono approcci più evoluti.

Overfitting e Rappresentazione Geometrica

Se riportiamo l'Albero di Decisione su un piano cartesiano (X = lunghezza petalo, Y = larghezza petalo), i suoi **Decision Boundaries** (confini di decisione) non saranno mai curve o linee oblique, ma solo **linee rette orizzontali e verticali** che tagliano lo spazio in tanti rettangoli.

L'Overfitting: Se non mettiamo limiti, l'albero continuerà a fare domande finché ogni singola foglia non è pura al 100%. Questo porta all'overfitting (il modello impara a memoria il training set ma va malissimo sul test set).

<https://colab.research.google.com/drive/1RGjvNQxRydbWV06rzOQCGXxmpBQKBqip#scrollTo=epPBo28YWXcT>

- Dal tuo test su *Colab/Jupyter*: se imposti *max_depth* = 5 (o superiore), l'albero scende troppo in profondità creando foglie con un numero bassissimo di samples (es. 1 o 3). L'albero sta creando regole assurde solo per isolare singoli fiori anomali.
- **Soluzione:** usare iperparametri come *max_depth* (profondità massima) o *min_samples_leaf* (minimo numero di campioni per creare una foglia) per forzare l'albero a rimanere generico.

Lezione 8 – 13/04/2026 –

<https://colab.research.google.com/drive/1jMMLg6dxv7tNyV6Z5f-IXCFtKWUuqYk#scrollTo=VvAaOUS0SKFa>

Apprendimento non supervisionato – Unsupervised Machine Learning

Nella maggior parte delle situazioni reali, i dati non arrivano con una "risposta corretta" o un'etichetta; in altre parole, la colonna **target** non esiste. In questi casi, possiamo scrivere delle imprecisioni, oppure affidarci all'apprendimento **Non Supervisionato**, che si occupa di trovare strutture e pattern intrinseci nei dati senza una guida esterna.

Il capo dell'Intelligenza Artificiale di Meta, Yann LeCun, utilizza questa famosa metafora: "Considerando tutti i problemi di Machine Learning, se si trattasse di una torta, la glassa è l'apprendimento supervisionato mentre il resto no"

Le applicazioni principali trattate dall'apprendimento non supervisionato sono:

- **Clustering:** raggruppamento di istanze simili
 - segmentazione dei clienti
 - analisi di immagini
- **Rilevamento di anomalie:** identificazione di eventi rari o sospetti
 - frodi bancarie
 - acquisti anomali
- **Stima della densità:** stima della distribuzione di probabilità dei dati
- **Apprendimento semi-supervisionato:** uso di dati non etichettati per migliorare le prestazioni di modelli supervisionati quando le etichette sono scarse
- **Operazione su due o più feature**

2. Classificazione vs Clustering

È fondamentale non confondere questi due compiti :

- **Classificazione (Supervisionata):** il modello impara a dividere le istanze in classi note basandosi su etichette fornite
 - specie di fiori Iris
- **Clustering (Non Supervisionato):** il modello raggruppa le istanze basandosi esclusivamente sulla loro posizione nello spazio e sulle loro similarità, senza conoscere i nomi delle classi. Il clustering **non distingue i nomi delle classi** a priori, è l'analista umano che, a posteriori, dà un senso ai gruppi emersi.

3. L'Algoritmo K-Means

Il K-Means è l'algoritmo di clustering più diffuso e si basa sul concetto di **centroide** (baricentro o centro di massa di un cluster).

- **3.1 Funzionamento e Geometria**

1. L'algoritmo posiziona casualmente **k** centroidi nello spazio dei dati.
2. Assegnazione: ogni istanza viene assegnata al centroide più vicino utilizzando la **distanza euclidea** $d = \sum_i \sqrt{(x_c - x_i)^2 + (y_c - y_i)^2}$ dove C è il centroide e i sono i punti nella nuvola
3. Aggiornamento: i centroidi vengono spostati al centro delle istanze appena assegnate
4. Il processo si ripete fino alla **convergenza**, ovvero quando i centroidi smettono di muoversi.

L'obiettivo del centroide è minimizzare la distanza euclidea, quindi cercare un baricentro tale per cui le distanze dei punti appartenenti a quel cluster siano minori rispetto alle distanze dai punti non appartenenti a esso.

Tassellazione di Voronoi: il risultato finale divide lo spazio in regioni (decision regions) dove ogni punto è più vicino al proprio centroide rispetto a qualsiasi altro. Un cluster è **robusto** se i suoi punti sono lontani dai confini della tassellazione. Nel testing-set, aggiungendo nuovi punti, se questi cadono lontano dal confine allora la clusterizzazione è robusta; se cadono vicino al confine, è difficile affermare con certezza a quale cluster appartenga il nuovo punto.

Il *k*-Means converge velocemente in un numero piccolo di iterazioni. Questa convergenza è garantita matematicamente perché **la somma delle distanze di un centroide non può essere inferiore a 0** (≥ 0)

3. Generazione Dati, K-Means++ e il limite della "Pallottola d'Argento"

Di default, il K-Means esegue un **Hard clustering** (una clusterizzazione netta: un punto appartiene a un cluster o non ci appartiene affatto). Un miglioramento fondamentale è il **k-means++**, un algoritmo che seleziona dall'inizio centroidi distanti tra loro, migliorando i risultati.

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs # classe che produce il dataset

# 5 centri dei blob
blob_centers = np.array([[ 0.2,  2.3], [-1.5,  2.3], [-2.8,  1.8],
                          [-2.8,  2.8], [-2.8,  1.3]])
blob_std = np.array([0.4, 0.3, 0.1, 0.1, 0.1]) # sparpagliamento dei blob:
# se metto 0.1 in tutti ci sono i centroidi che mi aspetto
X, y = make_blobs(n_samples=2000, centers=blob_centers,
                  cluster_std=blob_std,
                  random_state=7)

k = 5 # 5 centroidi
kmeans = KMeans(n_clusters=k, n_init=10, random_state=42)
y_pred = kmeans.fit_predict(X)
```

3.2 Limiti e Varianti

- **k-Means++**: variante che seleziona i centroidi iniziali in modo che siano distanti tra loro, migliorando drasticamente la velocità di convergenza e la stabilità.
- **Il limite della forma**: k-Means lavora bene con “bolle” circolari (sferiche) (`make_blobs`), ma fallisce con forme allungate o complesse (`make_moons`) (mezze lune). Il k-Means fallisce perché **lavora creando una circonferenza di raggio minimo con il centroide al centro**. Per le mezze lune serve lavorare con un iperpiano (o con algoritmi basati sulla densità come il DBSCAN).

Non esiste un algoritmo “Silver Ballet” (pallottola d'argento) perfetto per ogni dataset.

4. Scegliere il numero ottimale di Cluster (k)

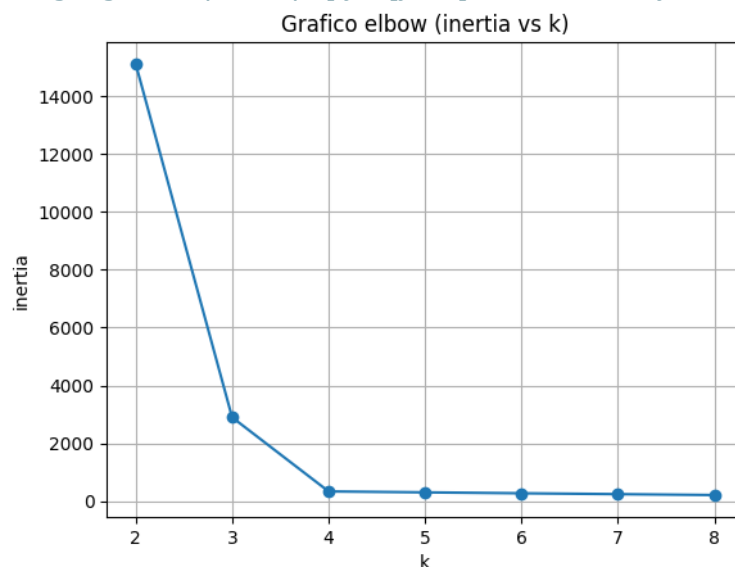
Se abbiamo un dataset reale (magari a 10 dimensioni) che non conosciamo, come facciamo a sapere quanti centroidi (k) creare? Lo facciamo a tentativi. Modificare k significa ottenere una clusterizzazione migliore o peggiore. Si usano due metodi principali di valutazione.

Metodo del Gomito (Elbow Method)

Si analizza l'**inerzia**, definita come la somma delle distanze al quadrato tra ogni istanza e il suo centroide più vicino.

- aumentando k , l'inerzia diminuisce sempre perché all'aumentare dei cluster, la distanza tra l'istanza e il proprio centroide diminuisce
- si traccia un grafico “Inerzia vs k ” e si sceglie il punto in cui la curva inizia ad appiattirsi (il “gomito” o elbow).

<https://colab.research.google.com/drive/1pyKqyxAqPLAiHYIRJdnJyU-6-RYXllvY>



in questo caso, il valore ottimale è $k = 4$

Attenzione: Il metodo del gomito è utile ma **non è precisissimo**.

Coefficiente di Silhouette

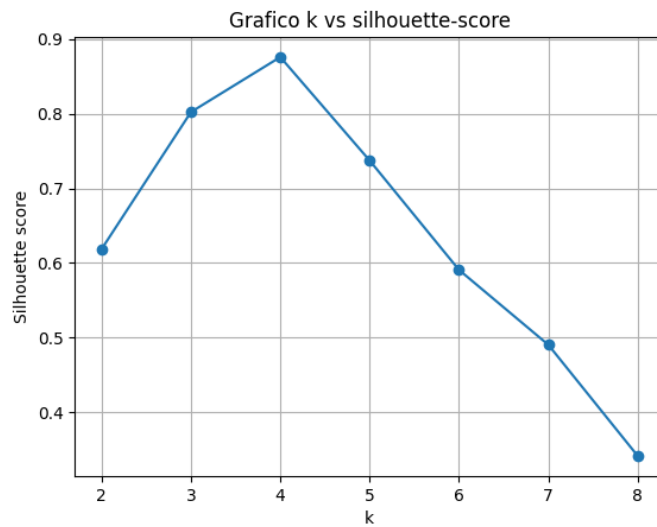
È una metrica più raffinata che valuta quanto un'istanza è vicina al proprio cluster rispetto a quelli vicini.

Formula: $s = \frac{b-a}{\max(a,b)}$

- **a**: distanza media dai punti dello *stesso* cluster.
- **b**: distanza media dai punti del cluster *più vicino*.

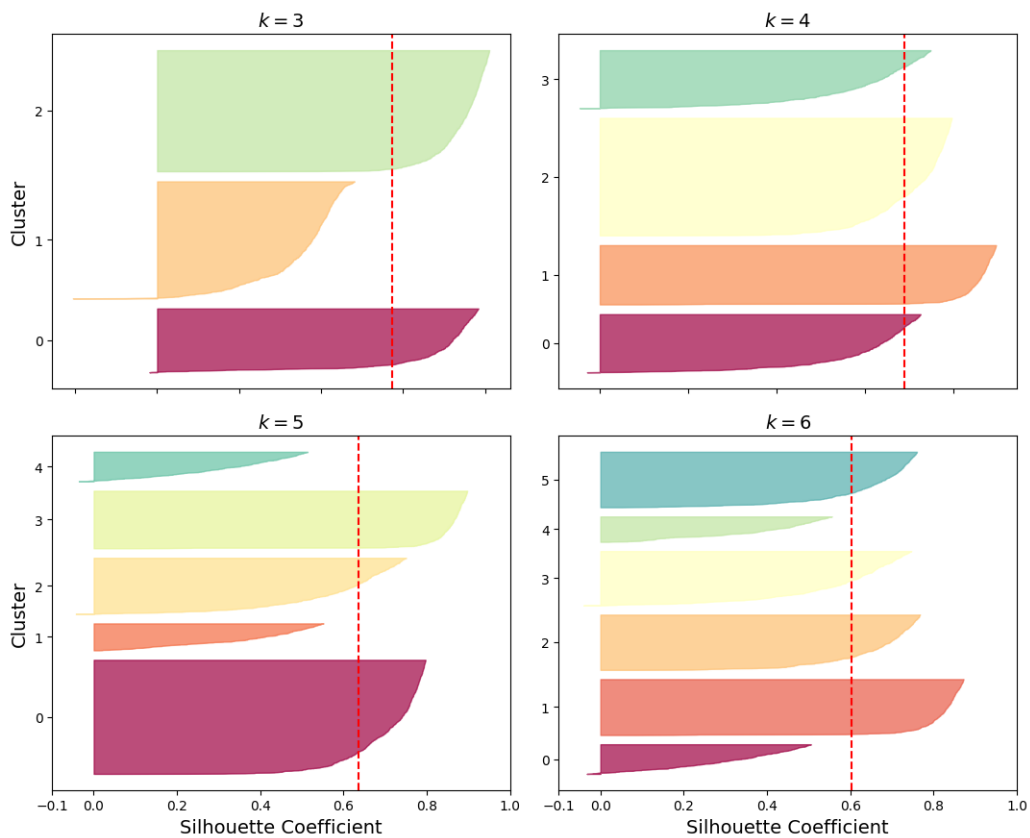
Interpretazione:

- $s \approx 1$: l'istanza è ben inserita nel suo cluster (ottima clusterizzazione).
- $s \approx 0$: l'istanza è vicina al confine tra due cluster (sul confine).
- $s < 0$: $a > b$: l'istanza è stata assegnata al cluster sbagliato.



in questo caso, il valore ottimale è $k = 4$

Grafico a coltello: visualizza i coefficienti di silhouette ordinati per ogni cluster. È un buon clustering se tutte le “lame” superano la linea rossa tratteggiata (media globale) e hanno dimensioni simili.



in questo caso, il valore ottimale è $k = 5$

Il Grafico (Silhouette Coefficient / Cluster):

- Ha una **forma di coltello**: i valori sono ordinati in modo decrescente (dalla più grande alla più piccola).
- Lo **spessore/altezza** della lama indica la quantità di istanze presenti in ogni cluster.
- La **linea tratteggiata verticale rossa** indica il valore medio. Questo valore medio *diminuisce all'aumentare del valore di cluster* (il valore di silhouette è inversamente proporzionale a k).
- *Conclusione della lezione*: osservando il grafico di silhouette, la migliore classificazione si ha con **$k=4$ o $k=5$** . È sempre necessario dare un'occhiata ai grafici per la proiezione nel piano prima di decidere.

I Limiti "Geometrici" del K-Means

Oltre al problema delle “mezze lune” (`make_moons`), i limiti del k-Means sono:

- densità e dimensioni diverse tra i cluster
- forme non sferiche che portano il k-Means a forzare la forma circolare commettendo degli errori

Lezione 9 – 14/04/2026 –

1. DBSCAN: Clustering Basato sulla Densità

A differenza del k-Means, che si basa sulle distanze dai centroidi, il **DBSCAN** (*Density-Based Spatial Clustering of Applications with Noise*) opera sulla **densità** delle nuvole di punti.

Concetti Chiave:

- **Core Instance (Istanza Centrale)**: un punto è considerato “core” se nel suo raggio ϵ (epsilon) cadono almeno un numero minimo di punti (`min_samples`).
- **In-layer vs Out-layer**: i punti interni al raggio epsilon sono definiti in-layer, mentre quelli esterni sono out-layer.

Vantaggi e Limiti:

- È ideale per dataset **non lineari** (come quelli generati da `make_moons`), dove le classi hanno forme a luna crescente che non possono essere divise da una semplice retta o piano.
- **Criticità**: se la densità è uniforme in tutto il dataset (senza zone più o meno affollate), l'algoritmo potrebbe fallire nel distinguere i cluster.

Previsioni: Scikit-learn non ha un metodo `predict()` nativo per DBSCAN. Per fare previsioni su nuovi dati, si addestra solitamente un classificatore secondario, come il **k-NN**, sui risultati ottenuti dal clustering.

Isolation Forest: Rilevazione delle Anomalie

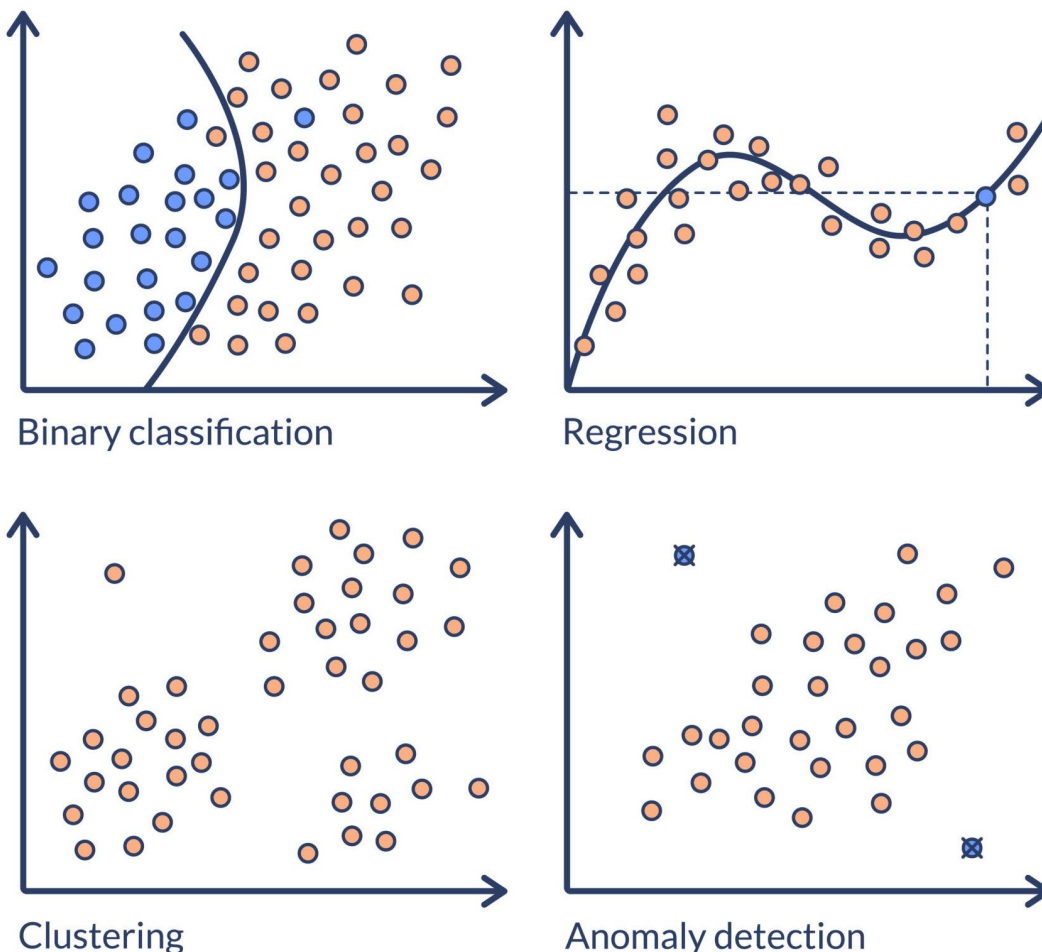
L'**Isolation Forest** è un algoritmo specifico per l'**Anomaly Detection**. Si basa sul principio che le anomalie sono “poche e differenti” rispetto alla massa dei dati.

- **Logica del “Taglio Random”**:
 - L'algoritmo seleziona un attributo a caso e lo divide (taglia) in un punto casuale.
 - **Isolamento rapido**: poiché le anomalie sono diverse dal resto, tendono a essere isolate in pochissimi passaggi.
 - **Profondità dell'albero**: più un dato viene isolato “vicino alla radice” (bassa profondità), più è probabile che sia un'anomalia.
- **Parametri Fondamentali**:

- o `n_estimators`: numero di alberi nella foresta. Se troppo basso, la varianza del modello aumenta.
- o `max_samples`: sottoinsieme di campioni affidati a ogni replica dell'albero.
- o `contamination`: la proporzione di outliers che ci aspettiamo di trovare
 - 0.01 se ci aspettiamo l'1% di anomalie

<https://stock.adobe.com/credit/images/machine-learning-tasks-graphic-scheme-of-a-classic-machine-learning-problem-binary-classification-regression-clustering-anomaly-search/539227026>

MACHINE LEARNING TASKS



3. Analisi Esplorativa (EDA) e Outliers

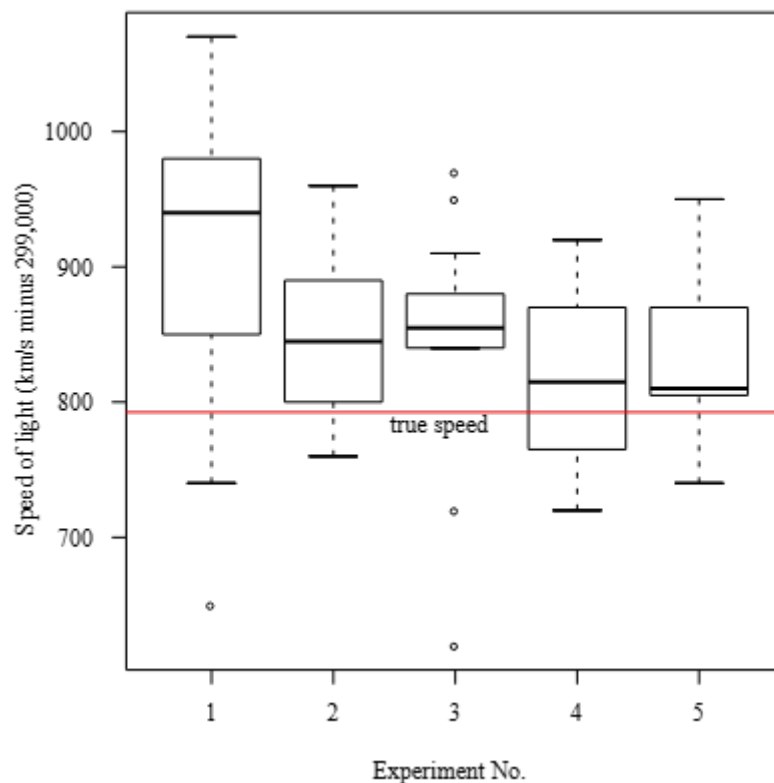
Per impostare correttamente i modelli (come il parametro `contamination`), è necessario analizzare la distribuzione dei dati tramite strumenti grafici.

Istogrammi e il fenomeno del "Capping": analizzando gli attributi numerici (dati immobiliari), si possono notare anomalie visive:

- **Mediana del guadagno:** solitamente segue una distribuzione normale.
- **Capping (Limitazione):** in alcuni dataset, i valori oltre una certa soglia vengono accorpati (es. tutte le case con più di 51 anni classificate come "51"). Questo crea un picco anomalo alla fine dell'istogramma.

Il Boxplot (Diagramma a scatola e baffi): è lo strumento standard per visualizzare la dispersione e identificare matematicamente gli outliers.

- **Struttura:**
 - **Q1 e Q3:** il primo e terzo quartile (25° e 75° percentile).
 - **Mediana:** dà linea che divide la scatola.
 - **IQR (Interquartile Range):** calcolato come $Q3 - Q1$
- **Calcolo degli Outliers:**
 - **Superiore:** $Valori > Q3 + (IQR \times 1,5)$
 - **Inferiore:** $Valori < Q1 - (IQR \times 1,5)$
 - I punti che cadono fuori da questi "buffer" (i baffi) sono rappresentati come pallini e classificati come anomalie.



4. Visualizzazione delle Decisioni

Nelle rappresentazioni 2D dei modelli di rilevazione:

- **Regione Azzurra:** Identifica i punti normali (in-layers).
- **Regione Rossa:** Identifica le zone dove il modello si aspetta di trovare gli **outliers**.
- I punti rossi sono i valori classificati effettivamente come anomali dalla funzione di decisione.

Lezione 10 – 20/04/2026 –

Reti Neurali Artificiali (ANN)

1. Introduzione e Applicazioni Moderne

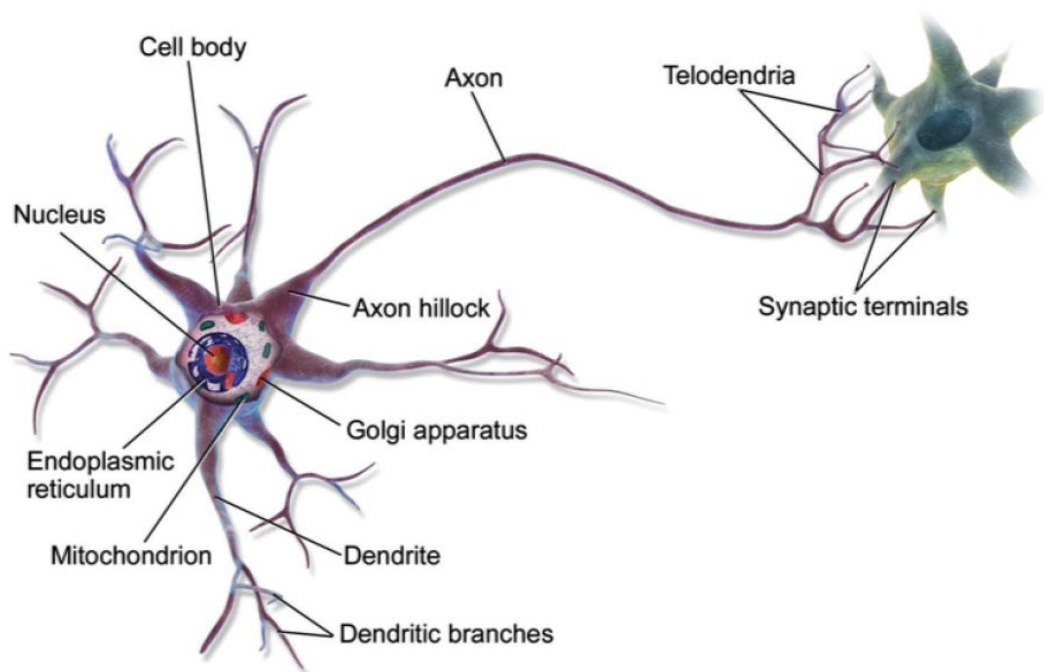
Le Reti Neurali Artificiali rappresentano oggi l'argomento *più caldo* dell'informatica. Non sono semplici algoritmi, ma strutture versatili capaci di affrontare:

- **Classificazione e Regressione:** identificare categorie o prevedere valori numerici.
- **Visione Artificiale:** riconoscimento di oggetti e volti.
- **Sistemi di Raccomandazione (Recommender Systems):**

- **Logica:** si basano sulla creazione di relazioni tra un insieme di **Utenti** e un insieme di **Prodotti** (struttura **ripartita**).
- **Profilazione:** piattaforme come Amazon generano liste dinamiche di suggerimenti analizzando utenti con caratteristiche simili alle nostre che hanno effettuato acquisti recenti simili.
- **Il Problema del Cold Start:** La difficoltà iniziale di suggerire prodotti a nuovi utenti (o nuovi prodotti a utenti esistenti) a causa della mancanza di dati storici. È vitale possedere molte informazioni sulle caratteristiche dei clienti (*customer*) per garantire incassi futuri.

2. Il Modello Biologico

L'ispirazione per le ANN deriva dai **neuroni biologici**, seguendo il principio per cui l'uomo si ispira alla natura (come il volo degli uccelli ha ispirato l'aeronautica), pur senza riprodurla esattamente.



Neural Network. Circuit Or Connections Of The Biological Neurons. Nervous System.

La struttura è caratterizzata:

- **anatomia del Neurone:** composto da:
 - un nucleo:
 - **biologicamente:** elabora i segnali ricevuto
 - **artificialmente:** è il nucleo del percettore (la TLU) dove avviene la somma pesata $z = w^T x + b$ e si applica la **funzione di attivazione**:
 - **attiva:** se il segnale supera la soglia, quindi vale 1
 - **disattiva:** se il segnale non supera la soglia, quindi vale 0
 - apparato del Golgi
 - **dendriti** (connessioni di ingresso):
 - **biologicamente:** tentacoli che ricevono segnali elettrici dai neuroni vicini
 - **artificialmente:** input determinati dal vettore dei pesi $w = \begin{pmatrix} w_1 \\ w_2 \\ \dots \\ w_n \end{pmatrix}$
 - un **assone** (connessione di uscita):

- **biologicamente:** è il “cavo” che trasporta l’impulso elettrico verso la cellula successiva
- **artificialmente:** è l’output finale del percettrone ed equivale alla predizione finale o al segnale passato al **DENSE LAYER** successivo.
- **il Cervello:** composto da circa neuroni. Gli informatici hanno imitato la loro distribuzione organizzandoli in **Layer** (strati) collegati, incaricati di trasmettere segnali a specifiche aree.

Di conseguenza, gli informatici imitano solo la logica di base dei neuroni biologici:

1. ricezione dei segnali da diverse fonti – i dendriti
2. valutazione e somma – nucleo
3. superamento della soglia di attenzione $\Rightarrow$ segnale di uscita – assone

3. Storia e Logica (Anni '50)

I primi neuroni artificiali (modello di McCulloch-Pitts) operavano come **bit** (attivo/spento) per eseguire operazioni logiche elementari:

- **Identità:** un neurone si attiva se e solo se raggiunto da due connessioni di input attive.
- **AND:** l’output si attiva solo se entrambi gli input sono presenti. Se manca la connessione A, la sola B non basta e viceversa.
- **OR:** ottenuta con due collegamenti attivi per ogni ingresso.
- **NOT (Inibizione):** un neurone A attivo combinato con un neurone B negato (connessione spenta) permette di implementare .

FOTO DELLE OPERAZIONI LOGICHE

4. Il Percettrone: L'Unità Logica a Soglia (TLU)

Il percettrone è il neurone artificiale più semplice. Riceve input, li pesa e decide se “sparare” un segnale in uscita.

4.1 La Matematica del Percettrone

Ogni input è associato a un peso al quale viene aggiunta una costante chiamata **Bias (b)**.

- **Pre-attivazione (z):** si calcola la somma pesata degli input più il bias.

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$
- **Notazione Vettoriale:** si usa la **trasposta (w^T)** perché il prodotto matrice per matrice richiede coerenza nelle dimensioni (vettore riga per vettore colonna). In questo stadio, il bias è uno **scalare**.

$$z = w^T x + b$$

Per la notazione vettoriale si usa la trasposta poiché sia i pesi che gli input nascono come vettori colonna:

$$w = \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix} \quad x = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$$

A causa della coerenza tra le dimensioni, bisogna trasformare il vettore dei pesi in un vettore riga, vale a dire eseguire la trasposta:

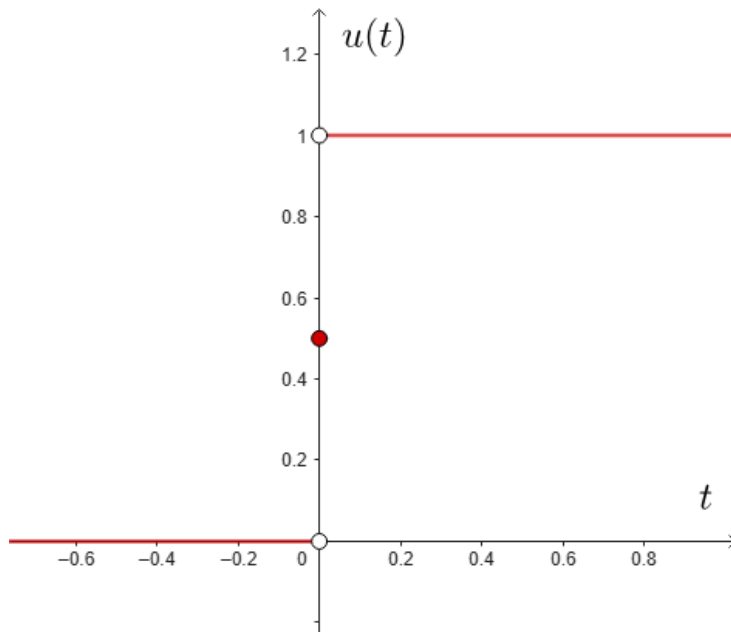
$$w^T = (w_1 \quad w_2 \quad w_3)$$

adesso, si può eseguire il prodotto in modo corretto.

4.2 La Funzione di Attivazione (Filtro)

Il perceptrone applica una **Funzione Soglia** (o **Funzione Gradino/Heaviside**) per restituire il risultato:

- se $z \geq 0 \Rightarrow \text{output}=1$:
 - il segnale si attiva
- se $z < 0 \Rightarrow \text{output}=0$:
 - il neurone scarta il segnale



CAMBIA GRAFICO

$$H(t) = \begin{cases} 0 & \text{se } t < 0 \\ 1 & \text{se } t \geq 0 \end{cases}$$

Questo significa che si ha un **confine netto** (decision boundary): funzione a tratti.

analogie del professore:

1. Udito

Immagina studiare in un bar affollato con tazzine che sbattono e persone che parlano: si tratta di input che entrano nelle orecchie ma la somma di tutti gli stimoli (z) non supera la “soglia di attenzione” del cervello. Di conseguenza, il risultato è 0: il cervello applica il filtro per evitare di far caso a quei rumori. Se all'improvviso qualcuno urlasse il nostro nome, quest'input avrebbe un peso altissimo tale che il risultato sia maggiore di 0 ($z \geq 0$), quindi l'output è 1 e avviene la focalizzazione su quel suono.

2. Vista:

Sei in una stanza con una luce normale e costante. Il nervo ottico manda il segnale al cervello, ma essendo un segnale piatto e continuo, il cervello lo declassa sotto la soglia. **Output = 0** (non ci fai più caso, ti abitui).

- Vai in discoteca e accendono le lampade stroboscopiche (flash fortissimi e improvvisi). Questo sbalzo di luminosità è uno stimolo che sfonda la soglia ($z \geq 0$).
- Il neurone visivo si attiva violentemente: **Output = 1**. Non puoi ignorare quel flash, la tua attenzione è catturata al 100%.

4.3 Il Ruolo del Bias

Il bias permette di traslare la funzione. Nell'equazione della retta, il bias è la (intercetta). Senza bias, la separazione tra le classi passerebbe sempre per l'origine, limitando drasticamente il modello. La **Funzione Segno ()** è un'alternativa, ma presenta salti finiti e non sempre può essere applicata all'origine.

5. Architettura Dense e Hardware (NVIDIA)

Quando raggruppiamo più TLU nello stesso strato, otteniamo un **Dense Layer** (o Full Layer).

- **Matrice dei Pesi:** Ogni input è collegato a ogni neurone di output.
 - *Esempio:* Per 3 TLU con 2 input, la matrice dei pesi ha **6 elementi**.
 - Qui lo scalare diventa un **vettore di bias**.
- **La Formula del Layer:**

5.1 Parallelizzazione vs Sequenzialità

Il calcolo delle matrici è il punto di forza delle schede grafiche (**GPU NVIDIA**).

- **Parallelismo:** Il prodotto riga per colonna è intrinsecamente parallelizzabile. Ogni unità di calcolo si occupa di una riga/colonna simultaneamente.
- **Complessità:** Il costo passa da a grazie all'hardware specializzato.
- **Operazioni Sequenziali:** Operazioni come la **Sequenza di Fibonacci** o progressioni aritmetiche (senza formula generale) non possono essere parallelizzate perché ogni passo dipende dal precedente.

6. Apprendimento: Come "impara" la rete?

Inizialmente i pesi sono numeri **random**. Il perceptrone apprende confrontando la previsione con il valore reale e aggiornando i pesi errati.

6.1 Regola di Aggiornamento

- **(Learning Rate):** Il passo di apprendimento.
- **:** L'errore (differenza tra target e predizione). I pesi corretti restano invariati, quelli errati vengono corretti singolarmente fino alla convergenza (separazione delle classi).

7. Limiti e l'Inverno dell'IA

Il perceptrone è perfetto per la **classificazione binaria linearmente separabile** (es. Dataset Iris: *Setosa vs Non-Setosa* usando solo le dimensioni del petalo).

- **Il Problema Non Lineare:** Il perceptrone non può risolvere problemi dove le classi non sono divise da una retta (es. il problema dello XOR).
- **L'Inverno dell'IA:** La scoperta di questo limite portò a un arresto dei finanziamenti e della ricerca negli anni '70, finché non si passò alle reti multi-strato (MLP).



Note di Laboratorio (Python/Colab)

- **Operatore @:** Si usa per il prodotto tra matrici (es. $X @ W$).
- **Funzione zip():** Essenziale per iterare contemporaneamente su input e target durante il training.
- **Semplificazione:** Nel laboratorio avete trasformato il problema Iris da multiclasse a binario per permettere al perceptrone di convergere.

Lezione 11 – 21/04/2026 –

Lezione 12 – 27/04/2026 –